

EEE 416 (January 2026)

Microprocessor and Embedded Systems Laboratory

Final Project Report

Section: B1 Group: 06

PRIVACY-PRESERVING HUMAN PRESENCE AND MOTION DETECTION USING WI-FI CSI

Course Instructors:

Dr. Zabir Ahmed, Assistant Professor, Department of EEE, BUET
Sadman Sakib Ahabab, Assistant Professor, Department of EEE, BUET

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____

Full Name: Tahmid Mahmud Fahim

Student ID: 2106078

Signature: _____

Full Name: Abid Uz Zaman

Student ID: 2106080

Signature: _____

Full Name: Anayet Hossain Nayeem

Student ID: 2106082

Signature: _____

Full Name: Robayet Hossen Bapon

Student ID: 2106083

Table of Contents

1	Abstract.....	1
2	Introduction	1
3	Design	2
3.1	Problem Formulation (PO(b)).....	2
3.1.1	Identification of Scope.....	2
3.1.2	Literature Review.....	3
3.1.3	Formulation of Problem.....	4
3.1.4	Analysis	4
3.2	Design Method (PO(a)).....	5
3.2.1	Wireless Measurement Model	5
3.2.2	Alignment and Normalization.....	5
3.2.3	Windowing and Empty Baseline.....	6
3.2.4	Presence and Motion Features	7
3.2.5	Learning and Decision Rules	7
3.3	Circuit Diagram	8
3.4	Simulation Model	9
3.5	Hardware Design	9
3.6	Full Source Code of Firmware.....	10
4	Implementation	18
4.1	Description.....	18
4.2	Firmware Development and Throughput Correction.....	19
4.3	Live Calibration and Phone Interface	19
4.4	Initial Pose Estimation Attempt.....	20
5	Design Analysis and Evaluation	21
5.1	Novelty.....	21
5.2	Design Considerations (PO(c))	22
5.2.1	Considerations to public health and safety	22
5.2.2	Considerations to environment	22
5.2.3	Considerations to cultural and societal needs	22
5.3	Investigations (PO(d)).....	23
5.3.1	Design of Experiment	23
5.3.2	Data Collection	24

5.3.3	Results and Analysis	24
5.3.4	Interpretation and Conclusions on Data.....	28
5.4	Limitations of Tools (PO(e))	28
5.5	Impact Assessment (PO(f)).....	29
5.5.1	Assessment of Societal and Cultural Issues	29
5.5.2	Assessment of Health and Safety Issues	29
5.5.3	Assessment of Legal Issues	29
5.6	Sustainability Evaluation (PO(g)).....	29
5.7	Ethical Issues (PO(h)).....	30
6	Reflection on Individual and Team work (PO(i)).....	30
6.1	Individual Contribution of Each Member.....	30
6.2	Mode of Team Work.....	31
6.3	Diversity Statement of Team	31
6.4	Log Book of Project Implementation	31
7	Communication to External Stakeholders (PO(j))	34
7.1	Executive Summary.....	34
7.2	User Manual.....	34
7.2.1	Hardware and Software Preparation	34
7.2.2	Troubleshooting	36
7.3	Github Link.....	36
7.4	YouTube Link.....	36
8	Project Management and Cost Analysis (PO(k)).....	36
8.1	Bill of Materials	36
8.2	Timeline of Project Implementation	37
9	Future Work (PO(l)).....	37
10	References	38

1 Abstract

This project develops a camera-free system for detecting human presence and gross movement from Wi-Fi Channel State Information (CSI). One ESP32-S3 acts as a dedicated transmitter and access point, while two ESP32-S3 receivers measure the wireless channel and stream CSI to a laptop through USB serial links. The laptop aligns the receiver streams on a common host-time grid, extracts amplitude features and runs lightweight machine-learning models. A phone browser provides live status and START and STOP controls. The final presence and activity system uses neither video nor audio as an inference input.

The acquisition archive contains 39 paired recording sessions, 78 primary CSV files and 227,505 received-packet records. Phase 1 uses 30 sessions to distinguish EMPTY from PERSON using 31 statistical and empty-baseline features. Its selected random forest achieves 99.82% window accuracy and a macro F1 score of 0.9973 on 1,089 held-out windows, with two false positives and no missed person windows. Phase 2 uses a 33-session subset with 75 features to distinguish EMPTY, STATIC and MOVING. The hierarchical presence–motion classifier achieves 93.14% window accuracy and macro F1 of 0.9421 on 1,210 held-out windows. A direct three-class logistic-regression model, selected by validation performance, achieves 91.98% test accuracy. Both Phase 2 models correctly classify all ten test sessions by their most frequent predicted class.

These results demonstrate feasibility for the recorded room and held-out participant. They do not establish room-independent performance: overlapping windows are correlated, only three participants were recorded, and the weakest static session exposes sensitivity to subject and location. The report presents the hardware and firmware, signal processing, model selection, experimental results, live dashboard, development setbacks, engineering considerations and reproducible operating procedure. An initial attempt at full 3D pose estimation was unsuccessful and motivated the narrower, experimentally supported presence-and-motion objective.

Keywords: channel state information, ESP32-S3, Wi-Fi sensing, human presence, random forest, embedded systems, privacy.

2 Introduction

Room occupancy supports automation, space management and non-intrusive monitoring. A useful detector should recognize stationary as well as moving occupants without requiring a wearable. Cameras record identifiable visual content, while simple motion sensors may provide limited evidence of a motionless person. These trade-offs motivate sensing through changes in radio propagation.

Wi-Fi CSI describes the complex channel response at multiple frequency positions. Human bodies alter propagation through attenuation, reflection and scattering. A stationary person may change the channel profile relative to an empty room, while movement introduces temporal variation. The project therefore combines empty-baseline comparison with short-term fluctuation features. Two receivers provide distinct propagation views, although their gain over one receiver is not separately quantified.

A dedicated transmitter gives control over packet timing, channel and transmitter identity. ESP32-S3 receivers export CSI through the ESP-IDF interface [3], and a laptop performs processing and inference. The phone supplies the user interface. This arrangement separates embedded acquisition from host computation while retaining a fixed one-transmitter/two-receiver system.

The task is a complex engineering problem because its constraints interact. More packets can improve temporal sampling but overload serial output. Normalization suppresses scaling artifacts but can discard information. Correlated windows can make evaluation misleading, and an accessible dashboard creates network-security requirements. The solution combines embedded programming, wireless communications, signal processing, machine learning and interface design.

The objective is an evaluated prototype that distinguishes EMPTY from PERSON and then STATIC from MOVING. Its implemented features include queued CSI export, paired acquisition, field-wise normalization, subject-separated evaluation and startup calibration. The project belongs to EEE 416, Microprocessor and Embedded Systems Laboratory, at BUET [8]. The report connects its design and evidence to the template's program-outcome structure [9].

Table 1. Comparison of candidate sensing approaches

Approach	Useful property	Main design trade-off
Camera	Detailed visual scene information	Records identifiable appearance and depends on view
PIR	Simple, inexpensive motion-triggered sensing	Limited evidence of a motionless occupant
Wearable	Measurements associated with a wearer	Requires a worn and powered device
Dedicated radar	Purpose-built radio sensing	Requires a separate RF sensing platform
Wi-Fi CSI	Device-free channel measurements without images	Depends on geometry, calibration and environmental stability

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

The target environment is an indoor sensing region with fixed device positions. The hardware configuration remains one ESP32-S3 transmitter and two ESP32-S3 receivers throughout both final project phases. The transmitter supplies a controlled packet stream. Both receivers connect to the same laptop, which runs all feature extraction and inference. A phone acts as a browser client and does not supply sensing data.

Phase 1 produces a binary output, EMPTY or PERSON. Phase 2 adds a coarse activity output, STATIC or MOVING, while retaining EMPTY as a possible overall state. STATIC includes standing at marked positions and sitting at the center. MOVING includes normal walking, slow walking, left–right walking and movement along the transmitter–receiver direction.

These are broad operational classes, not estimates of posture, identity, person count, vital signs or individual joint coordinates.

The evaluated data represent one room, three participant identifiers and approximately one-minute sessions. The study assesses generalization from training participant S01 to validation participant S02 and test participant S03 within the recorded arrangement. Empty-room sessions are separately allocated to the three partitions. Multi-room deployment, multiple simultaneous people, falls, pets and safety-critical control remain beyond the validated scope.

3.1.2 Literature Review

Wang et al. introduced Person-in-WiFi, showing that Wi-Fi measurements can support fine-grained person perception through learned mappings [1]. This work is relevant because it connects changes in wireless propagation to human structure and movement. Its task and training approach are more demanding than the binary and coarse-activity outputs pursued here. The present project does not reproduce its benchmark or claim comparable pose-reconstruction capability.

Yan et al. extended this research direction to end-to-end multi-person 3D pose estimation in Person-in-WiFi 3D [2]. The paper provides motivation for exploring richer output representations, but successful reconstruction depends on a suitable measurement and training pipeline. In the present project, a preliminary CSI-to-pose attempt produced unstable skeletons. The implemented deliverable therefore concentrates on a smaller output space supported by the available recordings.

Espressif documents CSI configuration, receive callbacks and the interpretation of long-training-field buffers [3]. These interfaces provide the practical basis for acquiring CSI on the ESP32-S3. The receiver applies a transmitter-identity filter, copies CSI out of the Wi-Fi callback and delegates serial formatting to a separate task. The host then selects the active amplitude positions that correspond to the project's fixed packet layout.

For the initial pose-labeling attempt, MediaPipe supplied a video-landmark framework [4]. Its standard pose output contains 33 landmarks; the presentation's 17-joint representation is a project-specific representation rather than the native landmark count. The final presence-and-motion dataset does not require video labels. Machine-learning evaluation follows the principle that preprocessing fitted from data and model selection must not use the held-out test set [5].

Table 2. Relation between prior work and the implemented project

Source	Relevant contribution	Use in this project
Person-in-WiFi [1]	Learned fine-grained perception from Wi-Fi	Motivation and context for the initial objective
Person-in-WiFi 3D [2]	End-to-end multi-person 3D pose estimation	Context for the difficulty of detailed reconstruction
ESP-IDF CSI guide [3]	CSI APIs and buffer interpretation	Acquisition and decoding foundation
MediaPipe Pose [4]	Video-based landmark extraction	Preliminary labeling attempt only

scikit-learn evaluation guidance [5]	Separation of training and evaluation	Subject and session partitioning discipline
--------------------------------------	---------------------------------------	---

3.1.3 Formulation of Problem

Let each receiver provide a sequence of timestamped complex channel measurements. After filtering and preprocessing, a two-second window from receiver r is represented by a matrix with 50 time samples and 107 retained amplitude values. The sensing problem is to estimate a class from the paired windows while preserving consistent feature definitions between offline evaluation and live operation.

The functional requirements are continuous dual-receiver acquisition, rejection of malformed or incompatible packets, alignment over common coverage, comparison with a valid empty-room baseline, reproducible classification and clear operator feedback. The operational requirements include recovery information when data are invalid, a configurable startup countdown, empty-room calibration and a phone interface that can start or stop inference.

Table 3. Requirements and implementation evidence

Requirement	Implementation	Evaluation evidence
Fixed sensing traffic	Dual unicast UDP every nominal 20 ms	Rates measured from the raw recordings
Traceable dual-RX records	Shared host clock and paired CSV filenames	39 primary recording pairs
Stable feature dimensions	Fixed HT20 layout and active-index selection	31 presence and 44 motion features
Held-out evaluation	S01 train, S02 validation, S03 test	Separate model-selection and test reports
Current room calibration	Baseline estimated at each live startup	Calibration stage in the final dashboard code
Operator control	Local HTTP service and phone browser	START, STOP, state and packet counters

3.1.4 Analysis

Presence and motion require related but different evidence. Comparing the mean spectral profile with an empty reference can detect a stationary obstruction. Temporal standard deviation, successive differences and low-frequency spectral variation help identify movement. A two-stage design makes the distinction explicit: the first classifier decides whether the room appears occupied, and the second examines motion features for an occupied window. A direct model instead assigns one of the three classes using all features together.

The two designs are evaluated separately. Validation selects the direct three-class model, whereas the final phone-control implementation uses the hierarchical presence–motion

structure. Both outcomes are retained in the report. Treating them as one model would incorrectly combine the validation decision with the deployed software architecture.

The most important data-integrity choice is to split complete sessions and participants before fitting data-dependent components. Adjacent windows overlap strongly and share most of their samples. Randomly distributing those windows among training and test partitions would create an overly easy evaluation. The empty baseline uses only the five training empty sessions, and the logistic-regression scaler is fitted inside its training pipeline.

3.2 Design Method (PO(a))

3.2.1 Wireless Measurement Model

For a transmitted symbol at frequency index k , the received signal can be represented by Equation 1. The channel response includes the combined effect of direct and reflected propagation paths, and the noise term includes receiver and environmental disturbances. Human presence can alter path attenuation and phase. The project uses the magnitude of the estimated complex channel rather than uncalibrated phase.

$$y_k(t) = H_k(t)x_k(t) + n_k(t) \quad (1)$$

The CSI byte pairs are stored as imaginary part followed by real part. Equation 2 converts a pair into amplitude. In the project's 256-byte layout, 128 bytes describe LLTF and the next 128 bytes describe HT-LTF, giving 64 complex positions in each field. These 128 positions are not 128 independent spatial channels: the two fields describe measurements of the same link using different training fields [3].

$$A_k = \sqrt{I_k^2 + Q_k^2} \quad (2)$$

The code keeps 51 LLTF values and 56 HT-LTF values. LLTF normally has 52 active positions in this layout, but the +1 position is conservatively excluded for every packet together with the DC position because the first CSI word may be invalid. The selection therefore provides a fixed 107-value representation without mixing valid and potentially invalid dimensions. Guard and DC positions are not treated as informative subcarriers.

3.2.2 Alignment and Normalization

Both capture workers use the laptop's monotonic clock relative to one start time. Host timestamps align the streams without assuming synchronized microcontroller clocks. They include USB and scheduling delay, so this is common-time resampling rather than sample-exact radio synchronization. The interpolation grid is restricted to the overlap of the two recordings. Linear interpolation produces a 25 Hz sequence; a host-time gap above 0.25 s rejects the session or live window instead of creating a long artificial bridge.

LLTF and HT-LTF are normalized separately by their per-packet median amplitude. Equation 3 expresses this for one field. $F(k)$ denotes the retained positions in the same LTF field as k . The implementation substitutes a denominator of one when its magnitude is below 10^{-6} . Separate normalization prevents the relative amplitude scaling of the two training fields from dominating the classifier. It also means the detector primarily uses normalized spectral shape and its changes; it is not an absolute received-power estimator.

$$\tilde{A}_{r,t,k} = \frac{A_{r,t,k}}{\text{median}_{j \in F(k)}(A_{r,t,j})} \quad (3)$$

A Savitzky–Golay filter smooths each retained dimension along time, using a five-sample window, polynomial order two and interpolation at the edges [7]. At 25 Hz, five samples span 0.16 s from first to last sample. This smoothing suppresses short fluctuations while preserving local trends. The preprocessing is not a dedicated, fully specified anti-alias filter, which limits claims about high-frequency motion content after resampling.

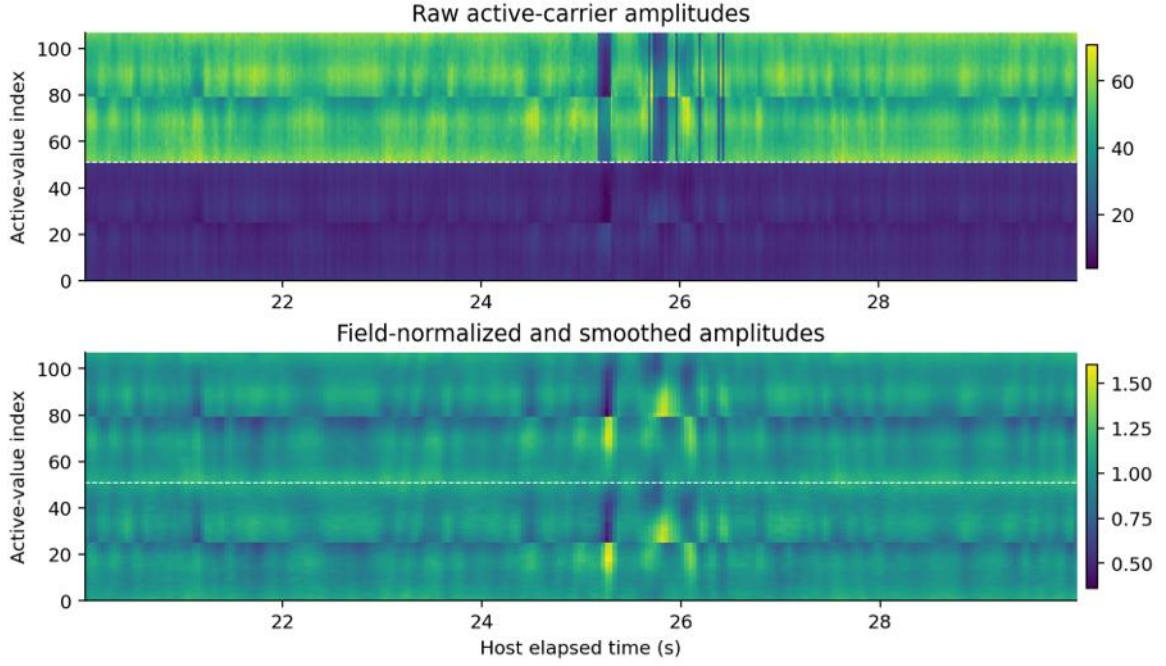


Figure 1. Recorded S01 transmitter–receiver walking data before and after preprocessing. The dashed separator distinguishes the 51 LLTF and 56 HT-LTF values. Color scales differ because the lower panel is normalized.

3.2.3 Windowing and Empty Baseline

Each feature window contains 50 samples, corresponding to a nominal two-second interval at 25 Hz. The requested offline step is 0.5 s. In the supplied implementation, $\text{int}(\text{round}(0.5 \times 25))$ evaluates to 12 samples, producing an actual offline hop of 0.48 s and 76% overlap. This explains the 121 windows obtained from each approximately 60 s session. The live loop is scheduled at a requested 0.5 s interval and prepares the latest complete common window.

$$N_{\text{windows}} = \text{floor}\left(\frac{N - 50}{12}\right) + 1 \quad (4)$$

The training empty-room reference is constructed separately for each receiver. First, a median profile is computed over time for each training empty session. A second median across the five session profiles gives the baseline. This reduces the influence of a single atypical training session. Validation and test empty recordings remain outside baseline fitting. Live operation estimates a new median profile during startup calibration while retaining the trained classifier weights.

3.2.4 Presence and Motion Features

The 31-dimensional Phase 1 vector contains 13 features from each receiver and five cross-receiver summaries. These describe average spectral shape, temporal variation, successive-sample change and deviation from the empty baseline. Baseline deviation is quantified by absolute differences, root-mean-square error and profile correlation. The cross-receiver features compare or combine baseline and motion summaries from the two links.

The 44 additional motion features comprise 18 per receiver and eight cross-receiver values. They summarize temporal standard deviation, mean absolute differences, root-mean-square differences, peak-to-peak ranges and packet-to-packet change. A spectral descriptor is obtained from the demeaned window using a real FFT. The code averages squared FFT magnitudes divided by window length over the 0.5–5 Hz bins, then applies $\log(1 + \text{value})$. This is a relative feature in normalized units, not a calibrated physical power measurement.

$$D_{RMSE} = \sqrt{\frac{1}{K} \sum_{k=1}^K (\mu_k - b_k)^2} \quad (5)$$

Here the mean profile and empty baseline are compared over all $K = 107$ retained positions for each receiver.

Table 4. Feature groups and dimensions

Feature family	Per receiver	Cross-receiver	Total
Presence profile and baseline features	13	5	31
Additional motion features	18	8	44
Combined Phase 2 input	31	13	75

3.2.5 Learning and Decision Rules

Phase 1 compares a single-feature threshold with logistic regression and a random forest. The threshold uses the average absolute baseline deviation across receivers. Its selected validation threshold is 0.022511673. Logistic regression uses standardization, $C = 1$, balanced class weights, a maximum of 5,000 iterations and random state 42. The Phase 1 random forest uses 300 trees, maximum depth five, minimum leaf size one, balanced class weights and random state 42. Class balancing compensates for unequal training class counts [6].

The Phase 2 motion model compares logistic regression against a random forest with 400 trees and maximum depth seven, trained only on STATIC and MOVING windows and the 44 motion features. The direct three-class candidates use all 75 features. The selection criterion is validation macro F1, with balanced accuracy as a tie-break. The selected motion model is a random forest; the selected direct model is logistic regression. Model fitting uses training data only, and the saved selected models are not refitted on validation data before the documented test.

For the cascade, a presence probability of at least 0.5 leads to a person decision. The motion probability is then compared with 0.5 to choose STATIC or MOVING. The direct model uses its predicted class. Accuracy counts all correct windows, balanced accuracy averages recall

across classes, and macro F1 gives each class equal weight. These complementary metrics prevent the more numerous person windows from hiding errors on EMPTY.

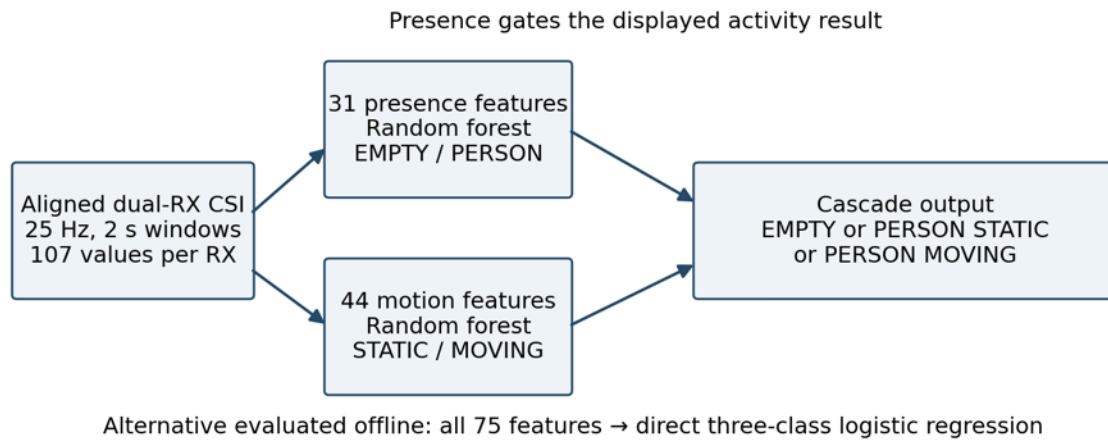


Figure 2. Feature allocation and the hierarchical classifier used by the phone implementation. The direct three-class alternative is evaluated separately.

3.3 Circuit Diagram

The electrical design is a connection-level prototype built from development boards. The transmitter receives low-voltage USB power from a power bank or suitable adapter. RX1 and RX2 each connect to the laptop through a data-capable USB cable, which supplies power and transports serial output. No external analog sensing circuit, inter-board GPIO bus or custom RF circuit is required. The sensing paths themselves are wireless.

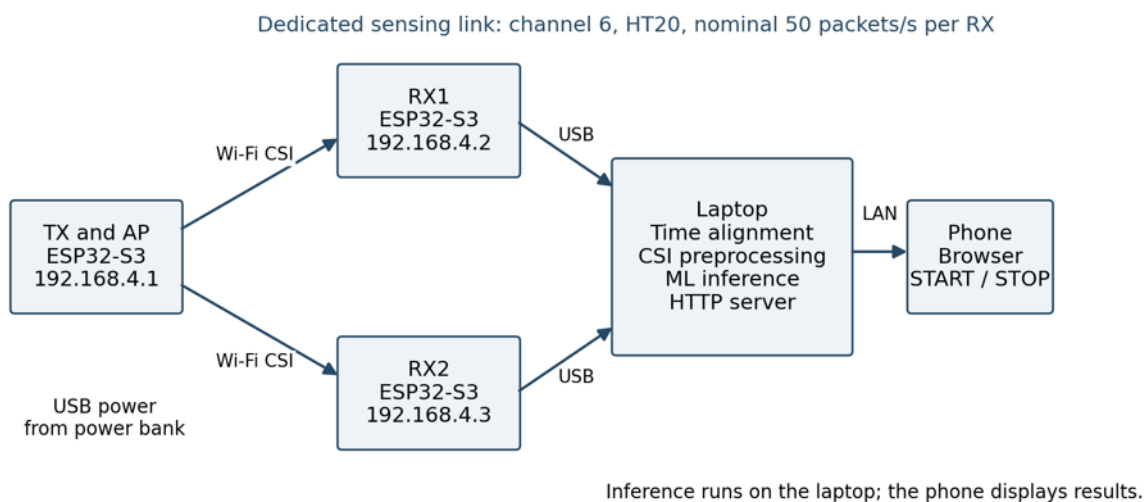


Figure 3. System connections and processing locations. The dedicated sensing network and the phone-to-laptop dashboard connection have different roles.

Table 5. Hardware and network configuration

Parameter	Implemented value
Development-board target	ESP32-S3 for TX, RX1 and RX2
Sensing network	TX access point and two receiver stations
Addresses	TX 192.168.4.1, RX1 192.168.4.2, RX2 192.168.4.3
Radio configuration	Channel 6, HT20, power saving disabled
Traffic	256-byte UDP payload to each receiver, port 3333
Nominal sending interval	20 ms per transmit cycle, approximately 50 packets/s per RX
Serial setting	921,600 baud
Host processing	Python, NumPy, pandas, SciPy, scikit-learn and joblib
Dashboard service	HTTP port 8080 by default

3.4 Simulation Model

The project is experimentally evaluated from acquired CSI rather than from an electromagnetic channel simulation. The offline software model represents the processing chain from paired raw recordings to feature vectors and classifier outputs. This model allows reproducible investigation of preprocessing, class separation and prediction errors using the same measurement archive.

The initial pose work also used learned mappings from CSI to a 17-joint output representation. That exploration is reported as an unsuccessful development path in Section 4.4, without attributing a validated pose accuracy to the final system. No CST, HFSS, ray-tracing or circuit-simulation result is claimed for the prototype.

3.5 Hardware Design

The transmitter was positioned on a stand, and receivers were attached at two shelf-side locations. The laptop remained connected to both receivers. Markers labeled C, L, R, TS and RS made the center, left, right, transmitter-side and receiver-side positions repeatable. Sitting was recorded at C. The device positions were held fixed between final collection sessions, while the participant or walking path changed [10].

Mechanical positioning is part of the signal design because antenna orientation, nearby surfaces and the person's path affect multipath propagation. The prototype uses available stands and supports. Dimensional manufacturing drawings are not required for this development-board assembly, but a deployable version would require repeatable mounts, cable strain relief and a documented antenna clearance.



Figure 4. Experimental sensing area from the project presentation. The room arrangement, furniture and fixed device locations form part of the evaluated environment.

3.6 Full Source Code of Firmware

Complete transmitter (TX), receiver 1 (RX1) and receiver 2 (RX2) application source files, headers and CMake build files.

<pre>tx/CMakeLists.txt # The following five lines of boilerplate have to be in your project's # CMakeLists in this exact order for cmake to work correctly cmake_minimum_required(VERSION 3.16) include(\$ENV{IDF_PATH}/tools/cmake/project.cmake) project(tx) tx/main/CMakeLists.txt idf_component_register(SRCS "wifi_ap.c" "udp_sender.c" "main.c" INCLUDE_DIRS ".") tx/main/main.c #include "nvs_flash.h" #include "esp_err.h" #include "freertos/FreeRTOS.h" #include "freertos/task.h" #include "wifi_ap.h" #include "udp_sender.h" void app_main(void) { esp_err_t ret = nvs_flash_init(); if (ret == ESP_ERR_NVS_NO_FREE_PAGES </pre>	<pre>tx/main/wifi_ap.c continued #define WIFI_SSID "fahim" #define WIFI_PASS "***** " #define WIFI_CHANNEL 6 #define MAX_STA_CONN 8 static const char *TAG = "WIFI_AP"; void wifi_ap_init(void) { ESP_LOGI(TAG, "Initializing network..."); //----- // Initialize TCP/IP Stack //----- ESP_ERROR_CHECK(esp_netif_init()); ESP_ERROR_CHECK(esp_event_loop_create_default()); esp_netif_create_default_wifi_ap(); //----- // Initialize WiFi Driver //-----</pre>
--	--

<pre> ret == ESP_ERR_NVS_NEW_VERSION_FOUND) { ESP_ERROR_CHECK(nvs_flash_erase()); ESP_ERROR_CHECK(nvs_flash_init()); } wifi_ap_init(); vTaskDelay(pdMS_TO_TICKS(3000)); udp_sender_start(); while (1) { vTaskDelay(pdMS_TO_TICKS(1000)); } } tx/main/wifi_ap.h #ifndef WIFI_AP_H #define WIFI_AP_H void wifi_ap_init(void); #endif tx/main/wifi_ap.c #include "wifi_ap.h" #include <string.h> #include "esp_event.h" #include "esp_log.h" #include "esp_netif.h" #include "esp_wifi.h" </pre>	<pre> wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT(); ESP_ERROR_CHECK(esp_wifi_init(&cfg)); //----- // Configure Access Point //----- wifi_config_t wifi_config = { .ap = { .ssid = WIFI_SSID, .ssid_len = strlen(WIFI_SSID), .channel = WIFI_CHANNEL, .password = WIFI_PASS, .max_connection = MAX_STA_CONN, .authmode = WIFI_AUTH_WPA2_PSK } }; if (strlen(WIFI_PASS) == 0) { </pre>
---	--

<pre> tx/main/wifi_ap.c continued wifi_config.ap.authmode = WIFI_AUTH_OPEN; } //----- // Start WiFi //----- ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_AP)); ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_AP, &wifi_config)); ESP_ERROR_CHECK(esp_wifi_start()); //----- // Recommended for CSI //----- ESP_ERROR_CHECK(esp_wifi_set_bandwidth(WIFI_IF_AP, WIFI_BW_HT20)); ESP_ERROR_CHECK(esp_wifi_set_ps(WIFI_PS_NONE)); //----- // Information //----- ESP_LOGI(TAG, "====="); </pre>	<pre> tx/main/udp_sender.c continued #define UDP_PORT 3333 #define PACKET_SIZE 256 #define SEND_INTERVAL_MS 20 static const char *TAG = "UDP_TX"; static void udp_sender_task(void *pvParameters) { char packet[PACKET_SIZE]; memset(packet, 'A', sizeof(packet)); int sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_IP); if (sock < 0) { ESP_LOGE(TAG, "Cannot create socket"); vTaskDelete(NULL); } // Define BOTH RX targets const char *target_ips[] = {"192.168.4.2", "192.168.4.3"}; int num_targets = 2; struct sockaddr_in dest_addr[2]; for(int i=0; i<num_targets; i++) { dest_addr[i].sin_addr.s_addr = inet_addr(target_ips[i]); dest_addr[i].sin_family = AF_INET; dest_addr[i].sin_port = htons(UDP_PORT); } ESP_LOGI(TAG, "UDP Sender Started (Dual Unicast)"); ESP_LOGI(TAG, "Targets: %s, %s Port: %d Interval: %d ms", target_ips[0], target_ips[1], UDP_PORT, SEND_INTERVAL_MS); </pre>
--	--

<pre> ESP_LOGI(TAG, "WiFi Access Point Started"); ESP_LOGI(TAG, "SSID : %s", WIFI_SSID); ESP_LOGI(TAG, "Password : %s", WIFI_PASS); ESP_LOGI(TAG, "Channel : %d", WIFI_CHANNEL); ESP_LOGI(TAG, "Max STA : %d", MAX_STA_CONN); ESP_LOGI(TAG, "Bandwidth: 20 MHz"); ESP_LOGI(TAG, "====="); } tx/main/udp_sender.h #ifndef UDP_SENDER_H #define UDP_SENDER_H void udp_sender_start(void); #endif tx/main/udp_sender.c #include "udp_sender.h" #include <string.h> #include <sys/socket.h> #include <netinet/in.h> #include <arpa/inet.h> #include "esp_log.h" #include "freertos/FreeRTOS.h" #include "freertos/task.h" </pre>	<pre> while (1) { // Send the same packet to both RX boards instantly for(int i=0; i<num_targets; i++) { sendto(sock, packet, sizeof(packet), 0, (struct sockaddr *)&dest_addr[i], sizeof(dest_addr[i])); } vTaskDelay(pdMS_TO_TICKS(SEND_INTERVAL_MS)); } void udp_sender_start(void) { xTaskCreate(udp_sender_task, "udp_sender_task", 4096, NULL, 5, NULL); } rx1/CMakeLists.txt # The following five lines of boilerplate have to be in your project's # CMakeLists in this exact order for cmake to work correctly cmake_minimum_required(VERSION 3.16) include(\$ENV{IDF_PATH}/tools/cmake/project.cmake) project(rx1) rx1/main/CMakeLists.txt idf_component_register(SRCS "csi_receiver.c" "wifi_sta.c" "main.c" INCLUDE_DIRS ".") rx1/main/main.c #include "wifi_sta.h" #include "csi_receiver.h" void app_main(void) </pre>
<pre> rx1/main/main.c continued { // Connect RX1 to the dedicated TX/AP first. wifi_sta_init(); // Then enable CSI on the active Wi-Fi link. csi_init(); } rx1/main/wifi_sta.h #ifndef WIFI_STA_H #define WIFI_STA_H void wifi_sta_init(void); #endif rx1/main/wifi_sta.c #include "wifi_sta.h" #include <string.h> #include "freertos/FreeRTOS.h" #include "freertos/event_groups.h" #include "esp_wifi.h" #include "esp_event.h" #include "esp_log.h" #include "esp_netif.h" #include "nvs_flash.h" #define WIFI_SSID "fahim" #define WIFI_PASS "***** " #define RX_IP_A 192 #define RX_IP_B 168 #define RX_IP_C 4 #define RX_IP_D 2 // RX1 = 192.168.4.2 </pre>	<pre> rx1/main/wifi_sta.c continued xEventGroupSetBits(wifi_event_group, WIFI_CONNECTED_BIT); } void wifi_sta_init(void) { esp_err_t ret = nvs_flash_init(); if (ret == ESP_ERR_NVS_NO_FREE_PAGES ret == ESP_ERR_NVS_NEW_VERSION_FOUND) { ESP_ERROR_CHECK(nvs_flash_erase()); ret = nvs_flash_init(); } ESP_ERROR_CHECK(ret); wifi_event_group = xEventGroupCreate(); if (wifi_event_group == NULL) { ESP_LOGE(TAG, "Failed to create Wi-Fi event group"); return; } ESP_ERROR_CHECK(esp_netif_init()); ESP_ERROR_CHECK(esp_event_loop_create_default()); esp_netif_t *sta_netif = esp_netif_create_default_wifi_sta(); if (sta_netif == NULL) { ESP_LOGE(TAG, "Failed to create default STA netif"); return; } // Static IP for RX1. // TX/AP is 192.168.4.1, RX1 is 192.168.4.2. esp_netif_dhcpc_stop(sta_netif); </pre>

```

#define GW_IP_D      1      // TX/AP = 192.168.4.1

static const char *TAG = "WIFI_STA";
static EventGroupHandle_t wifi_event_group;

#define WIFI_CONNECTED_BIT BIT0

static void event_handler(void *arg,
                        esp_event_base_t event_base,
                        int32_t event_id,
                        void *event_data)
{
    if (event_base == WIFI_EVENT && event_id ==
WIFI_EVENT_STA_START) {
        ESP_LOGI(TAG, "Connecting to AP...");
        esp_wifi_connect();
    }
    else if (event_base == WIFI_EVENT && event_id ==
WIFI_EVENT_STA_DISCONNECTED) {
        ESP_LOGW(TAG, "Disconnected. Reconnecting...");
        xEventGroupClearBits(wifi_event_group,
WIFI_CONNECTED_BIT);
        esp_wifi_connect();
    }
    else if (event_base == IP_EVENT && event_id ==
IP_EVENT_STA_GOT_IP) {
        ip_event_got_ip_t *event = (ip_event_got_ip_t
*)event_data;

        ESP_LOGI(TAG,
"=====");
        ESP_LOGI(TAG, "Connected Successfully!");
        ESP_LOGI(TAG, "IP Address : " IPSTR, IP2STR(&event-
>ip_info.ip));
        ESP_LOGI(TAG,
"=====");
    }
}

```

```

esp_netif_ip_info_t ip_info = {0};
ip_info.ip.addr      = ESP_IP4TOADDR(RX_IP_A, RX_IP_B,
RX_IP_C, RX_IP_D);
ip_info.gw.addr      = ESP_IP4TOADDR(192, 168, 4,
GW_IP_D);
ip_info.netmask.addr = ESP_IP4TOADDR(255, 255, 255, 0);
ESP_ERROR_CHECK(esp_netif_set_ip_info(sta_netif,
&ip_info));

wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();

// CSI must be enabled both here and at build time with
// CONFIG_ESP_WIFI_CSI_ENABLED=y.
cfg.csi_enable = 1;

ESP_ERROR_CHECK(esp_wifi_init(&cfg));

ESP_ERROR_CHECK(esp_event_handler_instance_register(
WIFI_EVENT,
ESP_EVENT_ANY_ID,
&event_handler,
NULL,
NULL));

ESP_ERROR_CHECK(esp_event_handler_instance_register(
IP_EVENT,
IP_EVENT_STA_GOT_IP,
&event_handler,
NULL,
NULL));

wifi_config_t wifi_config = {0};
strncpy((char *)wifi_config.sta.ssid, WIFI_SSID,
sizeof(wifi_config.sta.ssid) - 1);
strncpy((char *)wifi_config.sta.password, WIFI_PASS,
sizeof(wifi_config.sta.password) - 1);

```

rx1/main/wifi_sta.c continued

```

wifi_config.sta.threshold.authmode =
WIFI_AUTH_WPA2_PSK;

ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA,
&wifi_config));
ESP_ERROR_CHECK(esp_wifi_start());

// Disable Wi-Fi power saving for stable CSI
timing.
ESP_ERROR_CHECK(esp_wifi_set_ps(WIFI_PS_NONE));

ESP_LOGI(TAG, "Waiting for connection...");

xEventGroupWaitBits(
    wifi_event_group,
    WIFI_CONNECTED_BIT,
    pdFALSE,
    pdTRUE,
    portMAX_DELAY);
}

```

rx1/main/csi_receiver.h

```

#ifndef CSI_RECEIVER_H
#define CSI_RECEIVER_H
void csi_init(void);

```

```

#endif

```

rx1/main/csi_receiver.c

```

#include "csi_receiver.h"

```

```

#include <stdio.h>
#include <string.h>
#include <inttypes.h>

```

```

#include "esp_wifi.h"
#include "esp_log.h"
#include "esp_err.h"

```

```

#include "freertos/FreeRTOS.h"
#include "freertos/queue.h"
#include "freertos/task.h"

```

rx1/main/csi_receiver.c continued

```

if (info == NULL || info->buf == NULL) {
    return;
}

const uint8_t *expected_bssid = (const uint8_t
*)ctx;

// Keep only CSI frames originating from our
dedicated TX/AP.
if (expected_bssid != NULL && memcmp(info->mac,
expected_bssid, 6) != 0) {
    return;
}

// Keep only HT / 802.11n CSI packets from our
dedicated UDP stream.
// sig_mode = 0 -> legacy / non-HT (typically AP
beacons, often len=128)
// sig_mode = 1 -> HT / 802.11n (our desired
sensing packets, currently len=256)
if (info->rx_ctrl.sig_mode != 1) {
    return;
}

// For our fixed ESP32-S3 HT20 setup, 384 bytes is
a safe maximum.
// Never silently truncate an unexpected CSI frame.
if (info->len > CSI_MAX_LEN) {
    dropped_packets++;
    return;
}

csi_packet_t pkt = {0};

pkt.seq                = seq++;
pkt.timestamp_us       = info->rx_ctrl.timestamp;
pkt.rssi               = info->rx_ctrl.rssi;
pkt.noise_floor        = info->rx_ctrl.noise_floor;
pkt.channel            = info->rx_ctrl.channel;
pkt.secondary_channel  = info-
>rx_ctrl.secondary_channel;

```

```

#define CSI_MAX_LEN      384
#define CSI_QUEUE_LEN    128
#define CSI_LINE_MAX     2400

static const char *TAG = "CSI";

static QueueHandle_t csi_queue = NULL;
static uint32_t seq = 0;
static uint32_t dropped_packets = 0;
static uint8_t tx_bssid[6] = {0};

typedef struct {
    uint32_t seq;
    uint32_t timestamp_us;
    uint32_t dropped;

    int8_t rssi;
    int8_t noise_floor;

    uint8_t channel;
    uint8_t secondary_channel;
    uint8_t sig_mode;
    uint8_t mcs;
    uint8_t cwbc;
    uint8_t stbc;
    uint8_t first_word_invalid;

    uint16_t len;
    int8_t buf[CSI_MAX_LEN];
} csi_packet_t;

static void wifi_csi_cb(void *ctx, wifi_csi_info_t *info)
{

```

```

    pkt.sig_mode      = info->rx_ctrl.sig_mode;
    pkt.mcs            = info->rx_ctrl.mcs;
    pkt.cwbc           = info->rx_ctrl.cwbc;
    pkt.stbc           = info->rx_ctrl.stbc;
    pkt.first_word_invalid = info->first_word_invalid ?
1 : 0;
    pkt.len            = info->len;
    pkt.dropped        = dropped_packets;

    memcpy(pkt.buf, info->buf, pkt.len);

    // Never block the Wi-Fi task.
    if (xQueueSend(csi_queue, &pkt, 0) != pdTRUE) {
        dropped_packets++;
    }
}

static void csi_print_task(void *arg)
{
    csi_packet_t pkt;
    static char line[CSI_LINE_MAX];

    while (1) {
        if (xQueueReceive(csi_queue, &pkt,
portMAX_DELAY) == pdTRUE) {

            int pos = snprintf(
                line,
                sizeof(line),

```

rx1/main/csi_receiver.c continued

```

        "CSI_DATA,% PRIu32 ",%" PRIu32
",%d,%d,%u,%u,%u,%u,%u,%u,%u,%u,%u,%u,% PRIu32 ",\"[\",
    pkt.seq,
    pkt.timestamp_us,
    pkt.rssi,
    pkt.noise_floor,
    (unsigned)pkt.channel,
    (unsigned)pkt.secondary_channel,
    (unsigned)pkt.sig_mode,
    (unsigned)pkt.mcs,
    (unsigned)pkt.cwbc,
    (unsigned)pkt.stbc,
    (unsigned)pkt.len,
    (unsigned)pkt.first_word_invalid,
    pkt.dropped);

    if (pos < 0 || pos >= (int)sizeof(line)) {
        continue;
    }

    for (int i = 0; i < pkt.len; i++) {
        int written = snprintf(
            line + pos,
            sizeof(line) - pos,
            \"%d%s\",
            pkt.buf[i],
            (i < pkt.len - 1) ? \",\" : \"\");

        if (written < 0 || written >=
(int)(sizeof(line) - pos)) {
            pos = -1;
            break;
        }

        pos += written;
    }

    if (pos < 0) {
        continue;
    }

    snprintf(line + pos, sizeof(line) - pos,
        \"%J\\n\");
    fputs(line, stdout);
}

```

rx1/main/csi_receiver.c continued

```

    tx_bssid[0], tx_bssid[1], tx_bssid[2],
    tx_bssid[3], tx_bssid[4], tx_bssid[5]);

    csi_queue = xQueueCreate(CSI_QUEUE_LEN,
sizeof(csi_packet_t));
    if (csi_queue == NULL) {
        ESP_LOGE(TAG, \"Failed to create CSI queue\");
        return;
    }

    // Do not use ESP_ERROR_CHECK here while debugging:
log exact failure.
    esp_err_t err = esp_wifi_set_csi_config(&config);
    if (err != ESP_OK) {
        ESP_LOGE(TAG,
            \"esp_wifi_set_csi_config failed: %s
(0x%x)\",
            esp_err_to_name(err),
            (unsigned)err);
        return;
    }
    ESP_LOGI(TAG, \"CSI configuration accepted\");

    err = esp_wifi_set_csi_rx_cb(wifi_csi_cb,
tx_bssid);
    if (err != ESP_OK) {
        ESP_LOGE(TAG,
            \"esp_wifi_set_csi_rx_cb failed: %s
(0x%x)\",
            esp_err_to_name(err),
            (unsigned)err);
        return;
    }

    err = esp_wifi_set_csi(true);
    if (err != ESP_OK) {
        ESP_LOGE(TAG,
            \"esp_wifi_set_csi(true) failed: %s
(0x%x)\",
            esp_err_to_name(err),
            (unsigned)err);
        return;
    }

    BaseType_t ok = xTaskCreate(
        csi_print_task,

```

<pre> } } void csi_init(void) { // ESP32-S3 / ESP-IDF 5.5.x CSI configuration. // This matches Espressif's standard legacy/HT CSI configuration. wifi_csi_config_t config = { .lltf_en = true, .htlftf_en = true, .stbc_htlftf2_en = true, .lftf_merge_en = true, .channel_filter_en = true, .manu_scale = false, .shift = false, }; // Learn the currently connected AP/TX BSSID automatically. wifi_ap_record_t ap_info = {0}; ESP_ERROR_CHECK(esp_wifi_sta_get_ap_info(&ap_info)); memcpy(tx_bssid, ap_info.bssid, sizeof(tx_bssid)); ESP_LOGI(TAG, "TX/AP BSSID: %02x:%02x:%02x:%02x:%02x:%02x", </pre>	<pre> "csi_print", 4096, NULL, 3, NULL); if (ok != pdPASS) { ESP_LOGE(TAG, "Failed to create CSI print task"); return; } ESP_LOGI(TAG, "CSI enabled successfully"); } rx2/CMakeLists.txt # The following five lines of boilerplate have to be in your project's # CMakeLists in this exact order for cmake to work correctly cmake_minimum_required(VERSION 3.16) include(\$ENV{IDF_PATH}/tools/cmake/project.cmake) project(rx2) </pre>
---	---

<pre> rx2/main/CMakeLists.txt idf_component_register(SRCS "csi_receiver.c" "wifi_sta.c" "main.c" INCLUDE_DIRS ".") rx2/main/main.c #include "wifi_sta.h" #include "csi_receiver.h" void app_main(void) { // Connect RX2 to the dedicated TX/AP first. wifi_sta_init(); // Then enable CSI on the active Wi-Fi link. csi_init(); } rx2/main/wifi_sta.h #ifndef WIFI_STA_H #define WIFI_STA_H void wifi_sta_init(void); #endif rx2/main/wifi_sta.c #include "wifi_sta.h" #include <string.h> #include "freertos/FreeRTOS.h" #include "freertos/event_groups.h" #include "esp_wifi.h" #include "esp_event.h" #include "esp_log.h" #include "esp_netif.h" #include "nvs_flash.h" #define WIFI_SSID "fahim" #define WIFI_PASS "***** " #define RX_IP_A 192 #define RX_IP_B 168 #define RX_IP_C 4 #define RX_IP_D 3 // RX2 = 192.168.4.3 #define GW_IP_D 1 // TX/AP = 192.168.4.1 </pre>	<pre> rx2/main/wifi_sta.c continued ESP_LOGI(TAG, "====="); ESP_LOGI(TAG, "Connected Successfully!"); ESP_LOGI(TAG, "IP Address : " IPSTR, IP2STR(&event->ip_info.ip)); ESP_LOGI(TAG, "====="); xEventGroupSetBits(wifi_event_group, WIFI_CONNECTED_BIT); } void wifi_sta_init(void) { esp_err_t ret = nvs_flash_init(); if (ret == ESP_ERR_NVS_NO_FREE_PAGES ret == ESP_ERR_NVS_NEW_VERSION_FOUND) { ESP_ERROR_CHECK(nvs_flash_erase()); ret = nvs_flash_init(); } ESP_ERROR_CHECK(ret); wifi_event_group = xEventGroupCreate(); if (wifi_event_group == NULL) { ESP_LOGE(TAG, "Failed to create Wi-Fi event group"); return; } ESP_ERROR_CHECK(esp_netif_init()); ESP_ERROR_CHECK(esp_event_loop_create_default()); esp_netif_t *sta_netif = esp_netif_create_default_wifi_sta(); if (sta_netif == NULL) { ESP_LOGE(TAG, "Failed to create default STA netif"); return; } // Static IP for RX2. // TX/AP is 192.168.4.1, RX2 is 192.168.4.3. esp_netif_dhcpc_stop(sta_netif); esp_netif_ip_info_t ip_info = {0}; ip_info.ip.addr = ESP_IP4TOADDR(RX_IP_A, RX_IP_B, RX_IP_C, RX_IP_D); </pre>
--	---

<pre> static const char *TAG = "WIFI_STA"; static EventGroupHandle_t wifi_event_group; #define WIFI_CONNECTED_BIT BIT0 static void event_handler(void *arg, esp_event_base_t event_base, int32_t event_id, void *event_data) { if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_START) { ESP_LOGI(TAG, "Connecting to AP..."); esp_wifi_connect(); } else if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_DISCONNECTED) { ESP_LOGW(TAG, "Disconnected. Reconnecting..."); xEventGroupClearBits(wifi_event_group, WIFI_CONNECTED_BIT); esp_wifi_connect(); } else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP) { ip_event_got_ip_t *event = (ip_event_got_ip_t *)event_data; </pre>	<pre> ip_info.gw.addr = ESP_IP4TOADDR(192, 168, 4, GW_IP_D); ip_info.netmask.addr = ESP_IP4TOADDR(255, 255, 255, 0); ESP_ERROR_CHECK(esp_netif_set_ip_info(sta_netif, &ip_info)); wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT(); // CSI must be enabled both here and at build time with // CONFIG_ESP_WIFI_CSI_ENABLED=y. cfg.csi_enable = 1; ESP_ERROR_CHECK(esp_wifi_init(&cfg)); ESP_ERROR_CHECK(esp_event_handler_instance_register(WIFI_EVENT, ESP_EVENT_ANY_ID, &event_handler, NULL, NULL)); ESP_ERROR_CHECK(esp_event_handler_instance_register(IP_EVENT, IP_EVENT_STA_GOT_IP, &event_handler, NULL, NULL)); </pre>
---	--

<pre> rx2/main/wifi_sta.c continued NULL)); wifi_config_t wifi_config = {0}; strncpy((char *)wifi_config.sta.ssid, WIFI_SSID, sizeof(wifi_config.sta.ssid) - 1); strncpy((char *)wifi_config.sta.password, WIFI_PASS, sizeof(wifi_config.sta.password) - 1); wifi_config.sta.threshold.authmode = WIFI_AUTH_WPA2_PSK; ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA)); ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config)); ESP_ERROR_CHECK(esp_wifi_start()); // Disable Wi-Fi power saving for stable CSI timing. ESP_ERROR_CHECK(esp_wifi_set_ps(WIFI_PS_NONE)); ESP_LOGI(TAG, "Waiting for connection..."); xEventGroupWaitBits(wifi_event_group, WIFI_CONNECTED_BIT, pdFALSE, pdTRUE, portMAX_DELAY); } rx2/main/csi_receiver.h #ifndef CSI_RECEIVER_H #define CSI_RECEIVER_H void csi_init(void); #endif rx2/main/csi_receiver.c #include "csi_receiver.h" #include <stdio.h> #include <string.h> #include <inttypes.h> #include "esp_wifi.h" #include "esp_log.h" #include "esp_err.h" #include "freertos/FreeRTOS.h" #include "freertos/queue.h" #include "freertos/task.h" </pre>	<pre> rx2/main/csi_receiver.c continued uint16_t len; int8_t buf[CSI_MAX_LEN]; } csi_packet_t; static void wifi_csi_cb(void *ctx, wifi_csi_info_t *info) { if (info == NULL info->buf == NULL) { return; } const uint8_t *expected_bssid = (const uint8_t *)ctx; // Keep only CSI frames originating from our dedicated TX/AP. if (expected_bssid != NULL && memcmp(info->mac, expected_bssid, 6) != 0) { return; } // Keep only HT / 802.11n CSI packets from our dedicated UDP stream. // sig_mode = 0 -> legacy / non-HT (typically AP beacons, often len=128) // sig_mode = 1 -> HT / 802.11n (our desired sensing packets, currently len=256) if (info->rx_ctrl.sig_mode != 1) { return; } // For our fixed ESP32-S3 HT20 setup, 384 bytes is a safe maximum. // Never silently truncate an unexpected CSI frame. if (info->len > CSI_MAX_LEN) { dropped_packets++; return; } csi_packet_t pkt = {0}; pkt.seq = seq++; pkt.timestamp_us = info->rx_ctrl.timestamp; pkt.rssi = info->rx_ctrl.rssi; pkt.noise_floor = info->rx_ctrl.noise_floor; </pre>
--	--

```
while (1) {
```

```
err = esp_wifi_set_csi(true);
if (err != ESP_OK) {
    ESP_LOGE(TAG,
               "esp_wifi_set_csi(true) failed: %s
(0x%x)",
               esp_err_to_name(err),
               (unsigned)err);
    return;
}
```

<pre> "]\n"); fputs(line, stdout); } } void csi_init(void) { // ESP32-S3 / ESP-IDF 5.5.x CSI configuration. // This matches Espressif's standard legacy/HT CSI configuration. wifi_csi_config_t config = { .lltf_en = true, .htlftf_en = true, .stbc_htlftf2_en = true, .lftf_merge_en = true, .channel_filter_en = true, .manu_scale = false, .shift = false, }; // Learn the currently connected AP/TX BSSID automatically. wifi_ap_record_t ap_info = {0}; ESP_ERROR_CHECK(esp_wifi_sta_get_ap_info(&ap_info)); </pre>	<pre> BaseType_t ok = xTaskCreate(csi_print_task, "csi_print", 4096, NULL, 3, NULL); if (ok != pdPASS) { ESP_LOGE(TAG, "Failed to create CSI print task"); return; } ESP_LOGI(TAG, "CSI enabled successfully"); } </pre>
--	--

4 Implementation

4.1 Description

The implementation separates time-sensitive radio handling from serial output and separates data collection from model training. The receiver callback accepts measurements from the connected transmitter's BSSID and requires HT packets. It copies metadata and the CSI buffer into a queue using a nonblocking send. A dedicated FreeRTOS task formats and prints queued packets. Queue failures and oversized packets increment a drop counter. This avoids performing the long formatting operation inside the Wi-Fi callback.

The receiver queue holds 128 entries, with up to 384 CSI bytes per entry and a 2,400-character output buffer. These capacities provide headroom for the chosen format but do not make the channel lossless. The firmware's sequence number is assigned after its transmitter and HT filters; therefore, consecutive numbers establish continuity among accepted callbacks, not delivery of every over-the-air packet. The host stores these diagnostics alongside the measurement.

Two Python capture workers read the receiver ports at 921,600 baud. Each accepted line becomes a row containing host time, receiver time, packet metadata and the CSI array. Both workers share the same host start reference. A configurable countdown precedes the requested 60 s collection period. The capture program writes one RX1 and one RX2 CSV per session, flushes the files and releases the serial ports when acquisition ends.

The preparation scripts use manifests rather than combining raw files across sessions. Each receiver pair is opened, filtered, aligned, normalized, smoothed and windowed independently. The resulting feature rows are then concatenated while preserving session, label and split fields. This makes it possible to trace a classification error back to a particular recording and prevents a feature window from crossing an artificial boundary between different activities.

Table 6. Main software components

Component	Responsibility
tx/main/wifi_ap.c and udp_sender.c	Dedicated access point and periodic unicast traffic
rx1 and rx2 main modules	Station connection, CSI queue and serial output
01_capture_dual_rx.py	Shared-time dual-port acquisition and CSV storage
presence/csi_utils.py	CSI decoding, alignment, baseline and presence features
04_prepare_dataset.py	Phase 1 manifest processing and feature splits
05_train_presence.py	Phase 1 candidate training and validation selection
phase2_utils.py	Additional motion feature extraction
03_prepare_phase2_dataset.py	Combined 75-feature dataset preparation
04_train_phase2_models.py	Motion and direct-model selection
08_phase2_phone_control_FINAL_NO_CASCADE_LAG.py	Integrated calibrated live cascade and phone interface

4.2 Firmware Development and Throughput Correction

The early implementation used a five-millisecond sending interval and broadcast traffic. The presentation records repeated receiver disconnections during development. Two changes addressed the acquisition bottleneck: the transmitter moved to dual unicast, and its interval increased to 20 ms. The final receiver architecture also uses a queue and a separate print task. These changes support a more manageable stream and protect the radio task from blocking on text formatting [10], [11].

The nominal payload capacity of a 921,600-baud UART using an 8N1 framing assumption is 92,160 bytes/s. The presentation estimates approximately 164 kB/s for full CSI text at 200 packets/s, which exceeds that capacity. At 50 packets/s, the corresponding estimate is about 41 kB/s, or 45% of capacity. These are throughput-budget calculations; the exact byte count depends on printed integer lengths and framing, and a native USB transport has different physical transfer characteristics.

The measured recording rates range from 41.67 to 50.02 packets/s. They confirm that the acquisition runs near the intended 50 Hz rate but also show that nominal timing is not guaranteed. Downstream interpolation and gap rejection therefore remain necessary. The data-quality assessment in Section 5.3 reports receiver continuity separately from radio delivery.

4.3 Live Calibration and Phone Interface

The final interface starts in IDLE. Pressing START requests confirmation that the sensing area will be empty. A five-second countdown gives the operator time to leave, followed by a default twelve-second calibration interval. Both receiver streams are checked for sufficient

common coverage and large gaps. Median normalized profiles then define the current empty-room baseline, after which the acquisition buffers are cleared and live inference starts on newly collected data.

The classifier weights remain fixed during startup calibration. Only the empty reference is replaced. Optional comparison against the training reference reports baseline drift using profile RMSE. This is useful for diagnosing environmental change, but baseline replacement is not equivalent to retraining or proving room-independent operation.

The live loop uses two-second windows and a requested 0.5 s inference step. Five recent binary decisions provide majority voting for presence and motion. The final revision computes motion as soon as the current raw presence decision indicates PERSON, so motion voting can proceed alongside presence voting. The phone displays activity when the stable presence state is PERSON. This avoids an unnecessary sequential delay from waiting for one full stable vote before beginning the second.

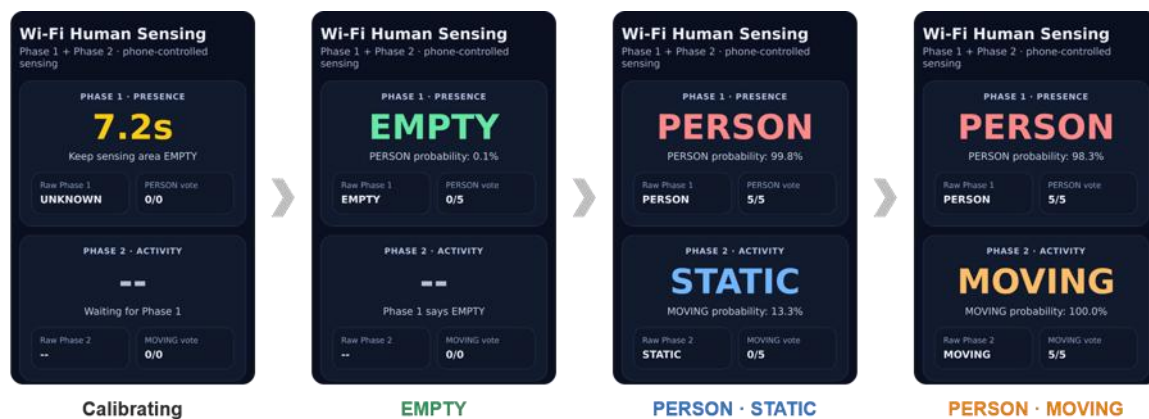


Figure 5. Phone-interface views from the presentation showing startup calibration, EMPTY, PERSON STATIC and PERSON MOVING. These views demonstrate interface states rather than a measured latency benchmark.

The dashboard is an HTTP service on the laptop. It exposes state information and START and STOP actions to a browser on a reachable local network. The phone does not need to join the dedicated transmitter network if it can reach the laptop through another local connection. The default bind address is 0.0.0.0 and the default port is 8080. The current implementation has no user authentication or HTTPS, so its intended setting is a controlled local demonstration network.

Invalid or stale receiver data prevent preparation of a new valid window and cause a waiting message. The display may retain the last classification while waiting. For that reason, an old label is not evidence of a fresh observation; the operator must also check the system message and receiver counters. Explicit UNKNOWN status and automatic expiration of old labels are deployment improvements.

4.4 Initial Pose Estimation Attempt

The initial target was to predict a 17-joint 3D human skeleton from dual-receiver CSI. Video-

based pose extraction supplied candidate labels during development, and CSI and video were paired by time. The intended inference stage would later use CSI alone. The presentation shows both plausible video-derived skeletons and unstable or distorted examples [10].

The attempted output contained 51 coordinate values per sample. This posed a substantially harder learning problem than detecting presence or gross motion. Only three people were recorded, some pose labels were unreliable, and imperfect temporal pairing could associate a CSI segment with an unsuitable target pose. Changes in the radio signal did not consistently translate into a reliable anatomical reconstruction.

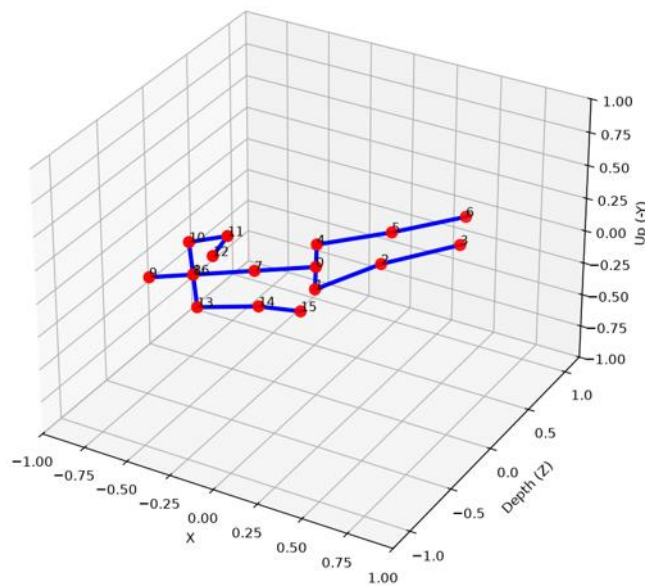


Figure 6. Example unstable skeleton prediction from the preliminary pose-estimation attempt. The example is qualitative development evidence and is not a validated 3D pose result.

The engineering response was to reduce the output complexity and collect explicit EMPTY, STATIC and MOVING recordings. This preserved the fixed one-transmitter/two-receiver hardware while shifting evaluation toward a tractable sensing objective. The failed attempt remains part of the project history because it explains the design decision and demonstrates the importance of matching model complexity to data quality and quantity. The final project does not claim accurate 3D pose estimation.

5 Design Analysis and Evaluation

5.1 Novelty

The project's contribution is a working integration of low-cost embedded CSI acquisition with reproducible host processing and a phone interface. The engineering features include dual-receiver records on a shared host time base, explicit PHY-layout handling, independent

normalization of the two LTF fields, a training-only empty baseline and startup recalibration. The final live revision also coordinates presence and motion voting to avoid an avoidable cascade delay.

These choices are useful within the project but do not constitute a claim that CSI sensing, random forests or hierarchical activity classification are new algorithms. The work demonstrates a complete prototype and reports its measured limits. Comparison with established research is therefore based on problem scope and implementation approach rather than unsupported claims of outperforming published pose-estimation systems.

5.2 Design Considerations (PO(c))

5.2.1 Considerations to public health and safety

The prototype uses low-voltage USB-powered development boards and does not switch mains loads. Cables and power banks must be secured so that recording paths remain free of trip hazards. Exposed board contacts should be kept clear of conductive surfaces and liquids. A suitable enclosure and cable strain relief are necessary for use outside the laboratory arrangement.

The sensing result is an experimental classification, not a certified safety signal. An EMPTY error could cause inappropriate automation, while a false PERSON result could waste energy or trigger an unnecessary alert. The present system should therefore provide advisory occupancy information. Emergency monitoring, fall detection and any action capable of affecting a person's safety require a separate validation and fail-safe design.

5.2.2 Considerations to environment

The system records numeric CSI and compact features instead of continuous video in its final operating mode. Development boards, USB cables and mounts can be reused across experiments. However, the laptop performs continuous acquisition and inference, and Wi-Fi power saving is disabled to maintain timing. Total energy efficiency cannot be inferred from the microcontroller alone. Section 5.6 evaluates this trade-off without assuming a measured power advantage.

5.2.3 Considerations to cultural and societal needs

The camera-free interface may suit spaces where visual recording is unwelcome. It also avoids requiring occupants to wear or charge a personal device. The dashboard uses explicit text states in addition to color, which helps interpret EMPTY, STATIC and MOVING. Occupants still need to know that radio-based monitoring is taking place because occupancy and movement are personal behavioral information even when no images are recorded.

5.3 Investigations (PO(d))

5.3.1 Design of Experiment

The data collection design holds the device arrangement fixed and varies participant, position and activity. Each primary session lasts approximately 60 s and is recorded simultaneously at both receivers. Three participants are identified as S01, S02 and S03. Nine empty-room sessions provide repeated reference conditions without a subject. The presentation documents floor markers and a five-second countdown used to make the procedure repeatable [10].

There are 39 unique primary sessions across the final archive. Phase 1 uses 30: nine EMPTY sessions and seven PERSON scenarios for each of three participants. The seven person scenarios are standing at the center, left, right, transmitter side and receiver side, sitting at the center, and normal walking. Phase 2 uses 33 sessions: all nine EMPTY recordings, four static scenarios per participant and four moving scenarios per participant. Its three added walking scenarios supply nine new sessions. The transmitter-side and receiver-side standing sessions remain Phase 1 data and are excluded from Phase 2. The phase totals overlap and must not be added as if they were independent experiments.

Table 7. Session membership and feature dimensions

Item	Phase 1	Phase 2
EMPTY sessions	9	9
Stationary-person sessions	18	12
Moving-person sessions	3	12
Total sessions used	30	33
Primary raw receiver files used	60	66
Total windows	3,630	3,993
Features per window	31	75

Training person data use S01, validation uses S02 and testing uses S03. The empty sessions are allocated as follows: training uses empty_01, empty_03, empty_05, empty_06 and empty_08; validation uses empty_02 and empty_07; testing uses empty_04 and empty_09. A session belongs to exactly one partition. The engineering_empty files under the earlier capture script directory are development recordings and are not part of these manifest-based datasets.

Table 8. Window counts in each evaluation partition

Phase and split	Sessions	EMPTY	PERSON or STATIC	MOVING	Total
Phase 1 training	12	605	847	—	1,452
Phase 1 validation	9	242	847	—	1,089

Phase 1 test	9	242	847	—	1,089
Phase 2 training	13	605	484	484	1,573
Phase 2 validation	10	242	484	484	1,210
Phase 2 test	10	242	484	484	1,210

5.3.2 Data Collection

Each raw file contains sixteen columns. The acquisition records the common PC timing reference, a receiver-local timestamp, a sequence counter, RSSI, noise-floor estimate, radio configuration, CSI length, invalid-word flag, firmware drop counter and CSI array. These metadata support diagnosis of transport and format problems instead of treating every line as an interchangeable numerical vector.

An audit of all 78 primary files gives 227,505 rows. File durations range from 59.8857 to 59.9780 s between first and last saved packet. The average packet rates for individual files range from 41.6718 to 50.0201 Hz. The largest host-time gap is 0.23615 s, below the processing limit of 0.25 s. There are no sequence gaps in the saved receiver counters, no nonzero firmware-drop counter values, and no recorded violations of the checked channel-6, HT20, non-STBC, 256-byte layout.

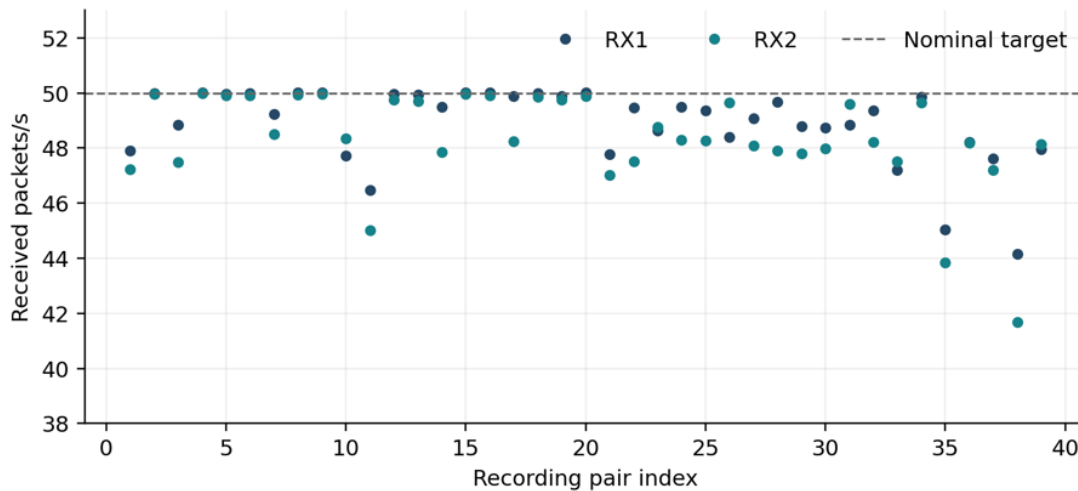


Figure 7. Measured packet rates for the 39 primary receiver pairs. The dashed line marks the nominal 50 Hz target; pair indices are only an inventory order.

These observations establish continuity of the saved accepted-packet streams. They do not prove zero wireless packet loss, because a packet that never reaches the accepted CSI callback does not receive the receiver's software sequence number. Similarly, a zero firmware-drop counter does not quantify interference from other transmitters. The observed rate and maximum gap are therefore reported alongside the counters.

5.3.3 Results and Analysis

The model bundles reproduce the saved held-out confusion matrices when evaluated against

their corresponding test feature tables. This consistency links the reported metrics to the preserved models and datasets. The results below use unrounded counts for calculations and display percentages to two decimal places.

The single-feature Phase 1 threshold achieves 93.20% validation accuracy and macro F1 of 0.9100. Its validation confusion matrix contains 238 correctly classified EMPTY windows, four false positives, 70 missed PERSON windows and 777 correct PERSON windows. The baseline-deviation feature is informative, but a single threshold does not cover the full variation represented by the multi-feature models.

Table 9. Phase 1 validation comparison

Model	Accuracy	Balanced accuracy	Macro F1	PERSON recall
Baseline-deviation threshold	93.20%	95.04%	0.9100	91.74%
Logistic regression	99.72%	99.82%	0.9960	99.65%
Random forest	99.91%	99.94%	0.9987	99.88%

Validation selects the random forest. On the held-out Phase 1 test set, it correctly predicts 1,087 of 1,089 windows. Its accuracy is 99.82%, balanced accuracy is 99.59%, PERSON precision is 99.76%, PERSON recall is 100%, and macro F1 is 0.9973. The two errors are EMPTY windows classified as PERSON in empty_09. All seven S03 person sessions have every window correctly classified for binary presence. Averaging the probability within each test session yields nine correct session decisions out of nine.

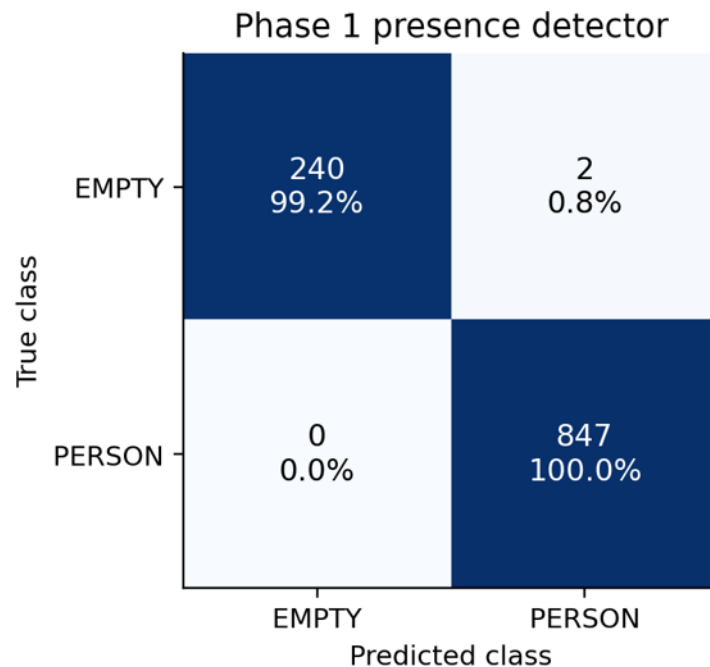


Figure 8. Phase 1 held-out test confusion matrix. Cells show the number of windows and the percentage within the true class. The test consists of 242 EMPTY and 847 PERSON windows.

For the person-only STATIC/MOVING classifier, validation selects the random forest with macro F1 of 0.9628, compared with 0.9451 for logistic regression. For the direct three-class

model, validation selects logistic regression with macro F1 of 0.9856, compared with 0.9817 for the random forest. The direct approach also exceeds the full cascade on validation, and the saved selection record identifies direct3 as the validation-selected deployment candidate.

Table 10. Phase 2 validation and held-out test comparison

Approach	Validation accuracy	Validation macro F1	Test accuracy	Test macro F1
Presence RF then motion RF	96.94%	0.9741	93.14%	0.9421
Direct three-class logistic regression	98.60%	0.9856	91.98%	0.9227

The cascade classifies 1,127 of 1,210 test windows correctly. Of its 83 errors, 65 are STATIC windows classified as MOVING, 16 are MOVING windows classified as STATIC, and two are EMPTY windows classified as a person state. Its per-class recall is 99.17% for EMPTY, 86.57% for STATIC and 96.69% for MOVING. The static–moving boundary is therefore the principal remaining classification weakness.

The direct classifier makes 97 test errors and correctly classifies all 242 EMPTY windows. It nevertheless predicts EMPTY for 18 STATIC and 13 MOVING windows, so its perfect EMPTY recall does not imply perfect occupancy detection. Its test accuracy falls by 6.61 percentage points from validation, compared with 3.80 points for the cascade. This is evidence of sensitivity to the held-out participant and session conditions, not a basis for changing the selection rule after examining the test set.

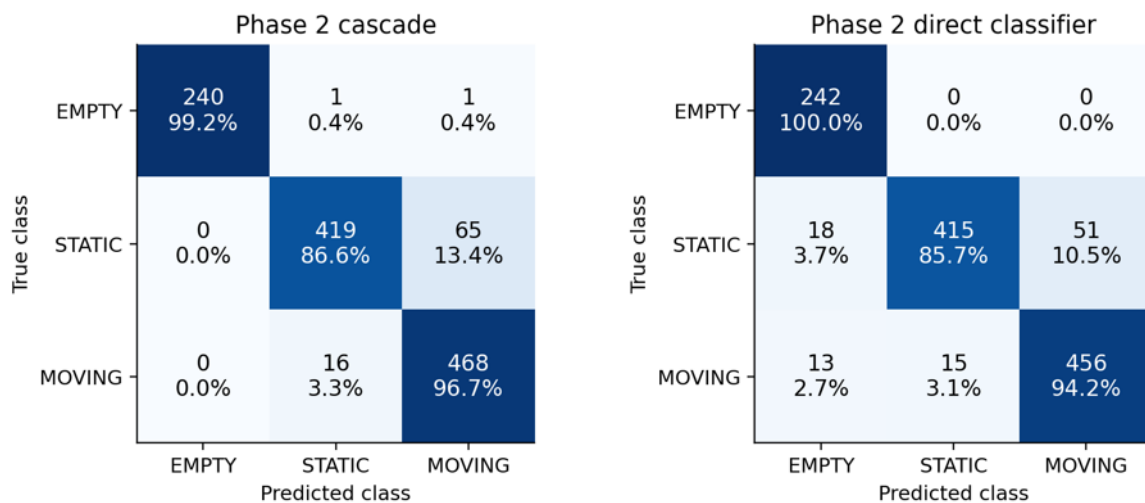


Figure 9. Phase 2 held-out confusion matrices for the cascade and direct models. Both use the same 1,210 test windows. Each cell gives its count and row percentage.

The integrated phone script implements the cascade, consistent with the demonstrated Phase 1/Phase 2 interface. The validation-selected direct model remains a separately evaluated alternative. The report therefore attributes 93.14% to the offline cascade and 91.98% to the direct classifier. Neither value is presented as a newly measured accuracy of the live dashboard with startup recalibration and voting.

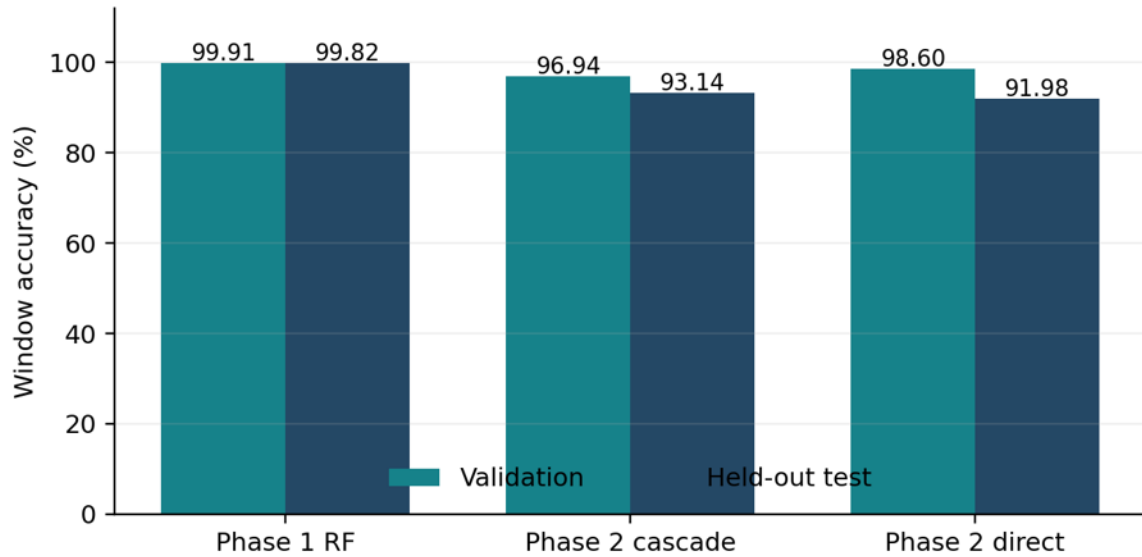


Figure 10. Validation and held-out test window accuracies. The Phase 1 result concerns binary presence, while the Phase 2 results concern three classes and are not identical tasks.

Session-level results require careful interpretation. The Phase 2 evaluator uses the most frequent predicted class within each recording, and both models correctly label all ten test sessions. This aggregation can hide difficult intervals. In particular, S03_stand_center has only 65 correct cascade windows out of 121, or 53.72%. It contributes 56 of the cascade's 65 STATIC-to-MOVING errors. The same session has 52 correct direct-model windows, or 42.98%, yet STATIC is still the most frequent of its three predicted classes. A correct session label therefore does not guarantee stable real-time classification throughout the recording.

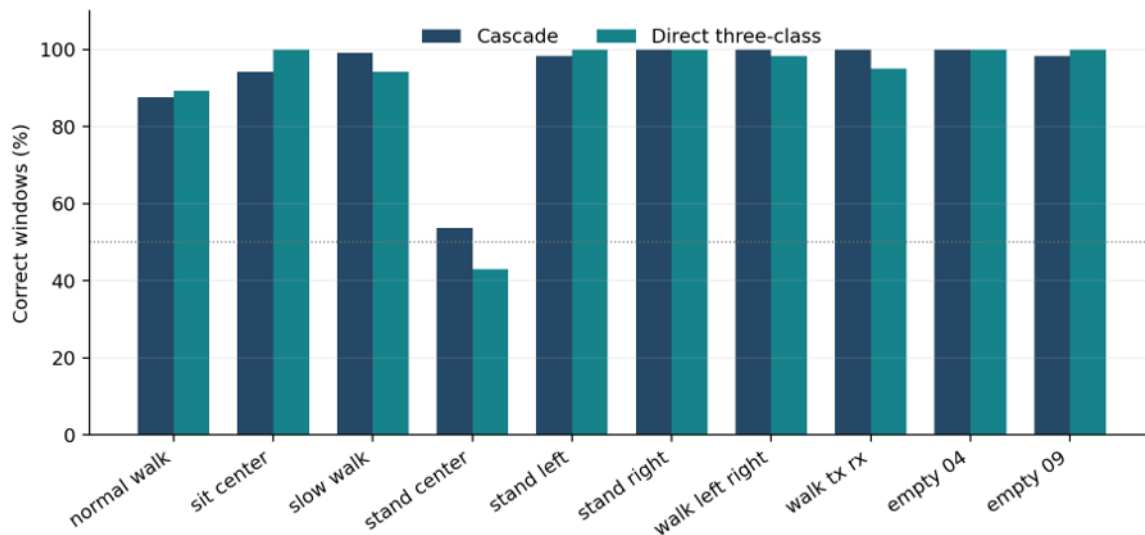


Figure 11. Per-session held-out window accuracy for the two Phase 2 approaches. Each bar summarizes 121 correlated windows. The standing-center session is the dominant failure case.

Possible explanations for this failure include small unlabelled movements, different body orientation or a location-dependent channel response. The available labels identify the intended session activity rather than every instantaneous movement, so the present evidence cannot select one explanation conclusively. A follow-up study should review time-resolved ground truth and repeat the same position with additional subjects before changing a classifier

threshold.

5.3.4 Interpretation and Conclusions on Data

The strongest supported finding is successful binary presence discrimination in the recorded room, including the held-out participant's standing and sitting sessions. Adding coarse motion recognition is feasible but introduces a substantial static–moving confusion that is concentrated in one position. The two-receiver design is operationally successful, but its superiority over one receiver requires an explicit ablation experiment.

The window count overstates the number of independent observations because successive windows overlap by 76% offline. The subject-separated split prevents the most direct form of adjacent-window leakage across partitions, but the study still has only one training person, one validation person and one held-out person. The same room and device arrangement also remain shared. Independent-session and independent-room repetitions are necessary before reporting general deployment reliability or uncertainty bounds based on independent samples.

The test set must remain an evaluation record. The observed errors can guide the design of a new collection campaign, but selecting or tuning a new model after inspecting S03 would require a new untouched test set. The present report preserves both model alternatives and identifies the implemented live architecture explicitly.

5.4 Limitations of Tools (PO(e))

CSI values depend on firmware settings, packet format and receiver processing. The fixed 256-byte interpretation is valid for the project's retained layout and must not be applied blindly to another bandwidth or CSI length. The firmware enables LTF merging and channel filtering, so the exported values already reflect receiver-side processing. Host filtering verifies the expected layout, but the generic offline loader selects the modal channel rather than universally enforcing channel 6. The collected archive itself contains channel 6 throughout.

Host timestamps include serial and operating-system delays. They support common-time interpolation but not precise radio-clock synchronization. A 25 Hz resampling rate limits the recoverable temporal content, and two-second windows limit frequency resolution to approximately 0.5 Hz. The classifiers infer statistical similarity to recorded conditions; they do not recover physical trajectories or anatomical motion.

The offline and live systems share decoding and feature definitions, but their boundary conditions differ. Offline smoothing operates on a full session and then slices windows, whereas live smoothing operates on the current window. The offline hop is 0.48 s after rounding, the live scheduler requests 0.5 s, and startup calibration replaces the baseline used during offline testing. These differences are reasons to measure live performance separately.

Model probabilities are classifier scores, not independently calibrated confidence guarantees. The five-decision vote can suppress isolated changes but can also delay a genuine state transition. The two-second window, scheduler, processing time and voting all contribute to response delay. A 0.5 s interface update must not be described as a measured 0.5 s detection latency.

5.5 Impact Assessment (PO(f))

5.5.1 Assessment of Societal and Cultural Issues

A camera-free occupancy system could support lighting or room-use information in privacy-sensitive spaces. Its benefit comes from limiting the detail collected while still providing the state needed by an application. This benefit depends on transparent installation and proportionate use. Occupants should be told what is measured, how results are used, who can view them and whether recordings are retained.

The system should not infer attendance quality, productivity or personal identity from the demonstrated outputs. Its error distribution also matters: a location where a stationary person is repeatedly classified as moving could produce misleading behavior summaries. Communication to users should retain the coarse, experimental meaning of the labels.

5.5.2 Assessment of Health and Safety Issues

The current implementation keeps electrical complexity low through development boards and USB power. Installation risk is dominated by practical issues such as unstable mounts, exposed contacts, damaged cables and obstacles along walking paths. These can be addressed by enclosure, inspection and secure routing. A deployed application also needs a safe response to disconnected receivers or stale results instead of assuming the last label remains valid indefinitely.

5.5.3 Assessment of Legal Issues

Deployment would require review of the applicable radio-equipment, network-access and personal-data requirements in the intended location. The prototype report does not assert regulatory certification. Permission to operate the sensing network and to monitor the space is distinct from the absence of a camera. Any external distribution should also preserve required software licenses and acknowledgements. The HTTP control service needs authentication and transport protection before exposure beyond a controlled local network.

5.6 Sustainability Evaluation (PO(g))

The development-board approach supports repair, reuse and replacement of individual components. The final dataset stores numeric radio measurements, and the live application can operate without recording video. Compact state logging could further reduce retention compared with keeping every raw packet, provided that detailed records are preserved when needed for controlled validation.

The system still consumes energy through three active radios, USB interfaces and a laptop. Its current power-saving configuration prioritizes consistent measurement timing. A valid energy comparison must measure the full operating system, including the host and power conversion, at a defined workload. The relationship between average power and energy is $E = P \times t$; for year-round operation, annual energy in kWh equals 8.76 times average power in watts. Every continuously saved watt would therefore reduce annual consumption by 8.76 kWh, but the prototype's actual power has not been established by this calculation.

Future efficiency work should test whether a smaller always-on host or on-device inference can retain acceptable detection quality. Reducing the sampling rate should follow a measured accuracy and latency comparison. End-of-life planning should favor reusable boards and recyclable enclosures, with electronic waste handled through suitable collection channels. These are design priorities rather than a completed life-cycle assessment.

5.7 Ethical Issues (PO(h))

The final sensing objective limits the output to occupancy and gross motion. The system does not need to identify a participant or reconstruct appearance. Nonetheless, a movement log can reveal behavior, so collection should be explained, participation should be voluntary, access should be limited and retention should serve a defined experimental or operational purpose. Participant codes in the dataset reduce unnecessary exposure of names but are not a substitute for consent.

The initial pose-labeling work involved video and therefore had a different privacy footprint from the final camera-free detector. These stages must be described separately. Calling the final inference camera-free does not erase the fact that video was used in preliminary development. Any retained pose-development recordings require their own handling and deletion decisions.

Honest performance reporting is also an ethical requirement. The report distinguishes validation from held-out testing, window decisions from session aggregation, intended timing from measured latency, and budget estimates from confirmed expenditure. It reports the failed pose attempt and the difficult standing-center session. The academic-honesty declaration is retained for the named authors to review and sign in accordance with the course template.

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contribution of Each Member

Student ID	Contribution
2106078	Contributed to the embedded hardware and firmware development of the project. Took primary responsibility for configuring the ESP32-S3 transmitter and receivers using ESP-IDF, setting the Wi-Fi parameters, enabling CSI acquisition, and implementing reliable CSI export through the receiver firmware. Also supported hardware setup, board flashing, communication testing, and troubleshooting of Wi-Fi and serial-data issues during system development.
2106080	Contributed to the CSI acquisition and preprocessing pipeline. Took primary responsibility for paired CSI data capture from the two receivers, recording-session management, packet validation, and preparation of the experimental dataset. Also worked on CSI decoding, receiver-stream alignment, resampling, normalization, and verification of the recorded data before feature extraction and model training.

2106082	Contributed mainly to the human-presence detection stage of the system. Developed Phase-1 feature engineering, prepared the empty-room baseline, and evaluated threshold-based, logistic-regression, and random-forest approaches for distinguishing EMPTY and PERSON states. Also performed model evaluation and testing and contributed to integration of the trained presence model with the live inference and calibration system.
2106083	Contributed mainly to the human-activity detection and user-interface stages. Developed motion-related features and machine-learning models for distinguishing STATIC and MOVING conditions. Also integrated the presence and motion classifiers into the hierarchical decision system, developed the local web server and phone-based monitoring interface, and supported final real-time system integration, voting logic, testing, and demonstration preparation.

6.2 Mode of Team Work

The division of tasks followed the processing chain, which created clear component ownership. Firmware produced a structured measurement stream; acquisition preserved that stream with timing metadata; preprocessing transformed paired recordings into consistent features; modeling produced saved artifacts; and the interface consumed those artifacts without redefining the feature schema. Each handoff could therefore be checked through a concrete file or interface.

The development setbacks show why integration must begin before all components are individually complete. Excessive serial output affected Wi-Fi stability, while model evaluation depended on reliable labels and recording boundaries. The final phone revision also required coordination between the two classifiers' state histories. The shared lesson is that a complete sensing system must be assessed across subsystem boundaries, including acquisition, inference and the user's interpretation of the displayed result.

6.3 Diversity Statement of Team

The team combined embedded programming, wireless configuration, data processing, statistical modeling and user-interface responsibilities. This technical diversity made it possible to approach faults from more than one discipline. The report attributes contributions by documented responsibility and does not infer demographic characteristics. Effective collaboration depended on understandable interfaces and the ability of each member to explain how their work affected the complete system.

6.4 Log Book of Project Implementation

Week	Project Topic / Assignment	Individual Role	Team Role	Comments
Week	Project Topic	All: Reviewed Wi-Fi CSI	All members	Project scope

3	Selection and Literature Review	sensing, ESP32-S3 CSI capability, privacy-preserving sensing and related previous works.	discussed possible project objectives and selected the initial project direction.	reviewed and initial objective selected.
Week 4	System Architecture and Hardware Planning	2106078: Studied ESP32-S3 firmware and CSI support. 2106080: Planned dual-RX CSI acquisition. 2106082: Studied presence-detection methods. 2106083: Studied activity-classification and interface requirements.	Team finalized the 1 TX + 2 RX + laptop + phone architecture.	Overall system architecture established.
Week 5	ESP-IDF Setup and CSI Acquisition	2106078: Configured ESP-IDF, TX/RX firmware, Wi-Fi settings and CSI callbacks. 2106080: Tested serial CSI capture. 2106082/83: Assisted testing and inspected CSI data.	Team tested communication among transmitter and two receivers.	CSI successfully captured from both receivers.
Week 6	Initial 3D Pose Estimation Attempt	All: Investigated CSI-based human pose estimation and experimented with video/MediaPipe pose labels and CSI synchronization.	Team evaluated the feasibility of reconstructing human pose from the available CSI data.	Pose estimation was unstable; project objective required modification.
Week 7	Project Scope Refinement	2106082: Developed EMPTY/PERSON presence-detection concept. 2106083: Developed STATIC/MOVING activity-classification concept. 2106078/80: Verified acquisition requirements.	Team redefined the final scope toward presence and gross activity detection.	Final project objective established.
Week 8	Firmware Development and Throughput Correction	2106078: Improved transmitter/receiver firmware, packet transmission and CSI export. 2106080: Checked packet continuity and serial data.	Team tested Wi-Fi stability, CSI rate and receiver operation.	Stable traffic configuration achieved.

Week 9	Dual-Receiver Data Collection	2106080: Managed paired CSI capture, session naming and validation. All: Collected EMPTY and PERSON recordings under controlled conditions.	Team maintained device placement and followed common recording procedures.	Primary presence dataset collected successfully.
Week 10	CSI Preprocessing and Baseline Preparation	2106080: Performed packet validation, decoding, alignment, resampling and normalization. 2106082: Prepared empty-room baseline and presence features.	Team inspected processed data and corrected invalid recordings.	Reliable aligned and normalized CSI dataset prepared.
Week 11	Presence Classification	2106082: Extracted presence features and evaluated threshold, logistic regression and random forest models. 2106080: Supported dataset preparation.	Team analyzed validation and test results.	EMPTY/PERSON presence detector successfully developed.
Week 12	Motion Data Collection and Activity Classification	2106080 & 2106083: Collected additional walking recordings. 2106083: Developed motion features and STATIC/MOVING classifier. 2106082: Assisted classifier integration.	Team tested static and different walking conditions.	Activity classification model completed.
Week 13	Live System Integration and Phone Interface	2106083: Integrated presence-motion cascade, web server and phone interface. 2106082: Integrated live presence model and calibration. 2106078/80: Supported hardware and CSI acquisition.	Team integrated acquisition, preprocessing, ML models and user interface.	Real-time EMPTY, PERSON STATIC and PERSON MOVING output achieved.
Week 14	Final Testing, Report and Demonstration	All: Performed final testing, prepared report, slides, figures and demonstration materials. Each member documented and presented their respective module.	Team jointly reviewed the complete project and delivered the final presentation.	Project completed and demonstrated successfully.

7 Communication to External Stakeholders (PO(j))

7.1 Executive Summary

This project detects room occupancy and broad human movement using Wi-Fi signals instead of cameras or microphones. One ESP32-S3 transmitter and two receivers send channel measurements to a laptop, which classifies the room as empty, occupied by a stationary person, or occupied by a moving person. A phone browser displays the result and controls operation. Recorded tests achieved 99.82% binary presence accuracy and 93.14% three-state accuracy for the cascade on held-out windows. The prototype requires an empty-room calibration and fixed device placement. It demonstrates practical camera-free sensing, while wider testing is needed before dependable deployment across different rooms and users.

7.2 User Manual

7.2.1 Hardware and Software Preparation

Use the three ESP32-S3 boards in their labeled TX, RX1 and RX2 roles. Place the transmitter and receivers in the evaluated arrangement, and keep the boards and surrounding furniture fixed while collecting or comparing sessions. Supply the transmitter through USB power. Connect both receivers to the laptop using data-capable cables. Do not run an ESP-IDF serial monitor and the capture application on the same receiver port simultaneously.

Install an ESP-IDF environment compatible with the archived ESP32-S3 projects. The presentation records ESP-IDF 5.5.5, and the source identifies the 5.5.x CSI configuration. Open each firmware folder independently and verify the target, CSI setting and serial setting before building. The archived sdkconfig files already select esp32s3; changing targets can regenerate configuration, so retain the supplied settings when rebuilding the same target [12].

Command sequence in each firmware project:

```
idf.py build
idf.py -p COM6 flash
idf.py -p COM6 monitor
```

COM6 is an example device port, not a fixed identifier. Use the port assigned to the board being flashed and repeat for TX, RX1 and RX2. Confirm that RX1 uses 192.168.4.2 and RX2 uses 192.168.4.3, and that all three firmware configurations use the same sensing-network credentials. Exit the monitor after checking connection and CSI output. The receivers must have CONFIG_ESP_WIFI_CSI_ENABLED=y and the console baud set to 921600.

Keep the extracted phase1 and phase2 directories adjacent under one project folder. In a Python environment, install the host requirements and list available serial ports:

```
python -m pip install -r phase2/requirements_phase2.txt
python phase1/python/00_list_ports.py
```

The dependency file lists NumPy, pandas, SciPy, scikit-learn, joblib and pyserial. Preserve the Python environment used to train saved joblib models, because model persistence is version-sensitive. When a model is incompatible with the local environment, rebuild it from the fixed training and validation feature tables using the documented scripts, then verify its

results before replacing an archived model.

Starting Live Operation

From the project root, launch the final phone-control application with the actual receiver ports. The following PowerShell example uses line-continuation backticks:

```
python phase2/python/08_phase2_phone_control_FINAL_NO_CASCADE_LAG.py `
--rx1 COM6 --rx2 COM8 `
--presence-model phase1/models/presence_model.joblib `
--motion-model phase2/models/motion_model.joblib `
--training-baseline phase1/data/processed/empty_baseline.npz `
--host 0.0.0.0 --port 8080
```

Connect the phone and laptop through a local network that permits phone-to-laptop access. Find the laptop's address on that network and open its HTTP address with port 8080 in the phone browser. For example, if the laptop address is 192.168.1.20, use <http://192.168.1.20:8080>. This example is the dashboard network address; it is separate from the receivers' fixed 192.168.4.x sensing addresses. Allow the application through the laptop firewall only for the intended local network.

Press START OPERATION, confirm the calibration instruction and leave the sensing region before the countdown finishes. Keep the region empty during calibration. After calibration completes, wait for a complete data window and check that both packet counters increase. Enter the region and verify the presence display, then stand or sit still and walk to observe the activity display. Interpret short transitions together with the raw state, vote information and system message.

Press STOP OPERATION before moving a board, changing the sensing area or terminating the demonstration. Restart and repeat calibration after changing device placement or the room. Calibration must not be performed while a person remains in the monitored region. To close the host service, stop operation first and then terminate the Python process from its terminal.

Recording a New Session

From the phase1 folder, the following example records a sixty-second receiver pair after the default countdown:

```
python python/01_capture_dual_rx.py --rx1 COM6 --rx2 COM8 `
--label new_subject_stand_center --duration 60 `
--countdown 5 --out data/raw
```

Use a unique session label and preserve both resulting files together. Record the intended activity, participant code, placement and unusual events. Validate each receiver's timing, metadata and continuity before adding a session to a manifest. Do not append unrelated recordings to a single raw CSV or assign windows from one session to different partitions. A new training campaign must reserve new held-out people or rooms rather than repeatedly tuning against S03.

7.2.2 Troubleshooting

Observation	Likely check	Action
Serial port will not open	Wrong port or another program owns it	List ports and close serial monitors or other capture scripts
Board is powered but no serial data	Charge-only cable or wrong interface	Use a known data-capable cable and the correct board port
Repeated receiver reconnects	TX power, credentials or traffic configuration	Check AP connection and the final 20 ms unicast firmware
One receiver counter stops	Disconnected cable, port error or receiver reset	Stop operation, restore the stream and recalibrate
Calibration fails	Area not empty, insufficient data or excessive gaps	Clear the area and check both acquisition streams
False presence after a room change	Baseline or geometry no longer representative	Restore placement or perform a new empty calibration
Phone cannot load the page	Network isolation, wrong laptop address or firewall	Verify local reachability and port 8080
Label remains during a waiting message	Last valid result is still displayed	Treat it as stale and inspect receiver counters before relying on it

7.3 Github Link

<https://github.com/Tahmid-Mahmud-Fahim/esp32-based-presence-and-motion-detection-using-wifi-csi>

7.4 YouTube Link

<https://youtu.be/RaYnB9Vjulw>

8 Project Management and Cost Analysis (PO(k))

8.1 Bill of Materials

Item	Quantity	Unit Cost (BDT)	Subtotal (BDT)
ESP32-S3 Development Board	3	750	2,250
Data-capable USB Cable	3	150	450
Nonmetal Enclosure / Stand Allowance	3	250	750
Labels, Cable Ties and Spare Parts	1	300	300

Laptop / Host Computer	Reused	0	0
Smartphone for Monitoring	Reused	0	0
Power Bank for Transmitter	Reused	0	0
Total Estimated Prototype Cost			3,750 BDT

8.2 Timeline of Project Implementation

Activity	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
Project Assignment												
Literature Review, Paper Study and Planning												
Establish Overall System Architecture												
ESP-IDF Firmware and CSI Acquisition												
Dual-RX Data Collection and Preprocessing												
Presence and Activity Model Development												
System Integration and Testing												
Documentation and Demo Preparation												
Final Presentation												

The primary management risks are loss of reproducibility, repeated tuning against the same test person, unstable room geometry and scope expansion into tasks unsupported by the data. Mitigation relies on preserving manifests and model versions, separating future test sets, recording device placement and requiring a measurable success criterion before adding new outputs. The failed pose-estimation attempt illustrates the cost of expanding output complexity before validating the measurement and label pipeline

9 Future Work (PO(I))

Prioritized Improvements

The first priority is broader validation of the existing outputs. A new dataset should contain more participants, repeated sessions on different days, multiple rooms and deliberate changes

in device placement. Tests should include long stationary occupancy and realistic nuisance conditions such as doors, fans and movement outside the intended region. Complete sessions and people must remain grouped during splitting, and a newly held-out set should be reserved after any changes motivated by the present results.

The second priority is a time-resolved live evaluation. Controlled entry, exit, standing-to-walking and walking-to-standing events should establish detection delay, false-alarm duration and stability after startup calibration. Receiver interruptions should test whether the interface correctly indicates stale data. Metrics should include event-based recall, false alarms per hour and delay distributions, in addition to window-level accuracy. Offline scores alone cannot establish these operating properties.

The third priority is engineering comparison. Single-receiver and dual-receiver models should be trained and evaluated with the same partitions to measure the value of the second receiver. Additional studies can isolate baseline normalization, LTF selection, window duration, sampling rate and voting length. Repeated subject- or session-grouped evaluations would clarify which decisions improve robustness rather than merely fitting the present room.

Deployment work should add authentication, protected transport, explicit freshness indicators and an UNKNOWN state during acquisition faults. Hardware packaging should establish repeatable antenna placement and protected power connections. Power should be measured at the complete-system boundary before selecting a smaller host or on-device inference. Advanced activity classes, multi-person sensing and pose reconstruction should follow only when the dataset and evaluation protocol can support them.

Conclusion

The project implements a complete Wi-Fi CSI sensing chain using one ESP32-S3 transmitter, two ESP32-S3 receivers, host-side feature extraction and a phone browser interface. The final acquisition architecture addresses serial-output pressure through a slower unicast stream and queued printing. The processing pipeline converts compatible paired measurements into normalized features while preserving session boundaries and subject-separated evaluation.

On the recorded held-out test set, the Phase 1 random forest achieves 99.82% binary presence accuracy. Phase 2 achieves 93.14% window accuracy with the hierarchical cascade and 91.98% with the validation-selected direct classifier. Correct session-level aggregation does not remove the observed standing-center weakness or establish uninterrupted real-time reliability. The camera-free presence-and-motion objective is supported by the experiment; detailed 3D pose estimation is not.

The final outcome is a reproducible laboratory prototype with clear hardware, software and operational documentation. Its next step is independent live and multi-environment validation, followed by security, freshness handling and packaging improvements appropriate to the intended deployment.

10 References

- [1] F. Wang, S. Zhou, S. Panev, J. Han, and D. Huang, “Person-in-WiFi: Fine-Grained Person Perception Using WiFi,” in Proceedings of the IEEE/CVF International Conference

on Computer Vision, 2019, pp. 5452–5461.

https://openaccess.thecvf.com/content_ICCV_2019/html/Wang_Person-in-WiFi_Fine-Grained_Person_Perception_Using_WiFi_ICCV_2019_paper.html

[2] K. Yan, F. Wang, B. Qian, H. Ding, J. Han, and X. Wei, “Person-in-WiFi 3D: End-to-End Multi-Person 3D Pose Estimation with Wi-Fi,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 969–978.

https://openaccess.thecvf.com/content/CVPR2024/html/Yan_Person-in-WiFi_3D_End-to-End_Multi-Person_3D_Pose_Estimation_with_Wi-Fi_CVPR_2024_paper.html

[3] Espressif Systems, “Wi-Fi Driver: Wi-Fi Channel State Information,” ESP-IDF Programming Guide, ESP32-S3, v5.5.1. <https://docs.espressif.com/projects/esp-idf/en/v5.5.1/esp32s3/api-guides/wifi.html>

[4] Google, “Pose landmark detection guide,” MediaPipe, Google AI Edge. https://developers.google.com/edge/mediapipe/solutions/vision/pose_landmarker

[5] scikit-learn developers, “Common pitfalls and recommended practices” and “Cross-validation: evaluating estimator performance,” scikit-learn documentation. https://scikit-learn.org/stable/common_pitfalls.html and https://scikit-learn.org/stable/modules/cross_validation.html

[6] scikit-learn developers, “RandomForestClassifier,” scikit-learn v1.8 documentation. <https://scikit-learn.org/1.8/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

[7] SciPy developers, “scipy.signal.savgol_filter,” SciPy API Reference. https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.savgol_filter.html

[8] Department of Electrical and Electronic Engineering, BUET, “EEE 416: Microprocessor and Embedded Systems Laboratory,” undergraduate course information. <https://eee.buet.ac.bd/academics/undergraduate/courses/eee416>

[9] Department of Electrical and Electronic Engineering, BUET, “Mission, PEO and PO of Dept of EEE, BUET,” program-outcome statements. <https://eee.buet.ac.bd/academics/undergraduate/obe/peo-po>

[10] Group B1-06, “Privacy-Preserving Human Presence and Motion Detection Using Wi-Fi CSI,” final project presentation, Group6-B1.pptx, EEE 416, BUET, 2026. Project photographs, contribution allocation, development history and baseline budget.

[11] Group B1-06, final.zip, electronic project archive, EEE 416, BUET, 2026. Firmware; Phase 1 and Phase 2 capture and processing scripts; manifests; raw and processed data; trained models; model-selection and test-results JSON files. Primary empirical source for this report.

[12] Espressif Systems, “Start a Project,” ESP-IDF Programming Guide, ESP32-S3, v5.5. <https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32s3/get-started/windows-start-project.html>

Online documentation and publication metadata accessed 1 October 2026. Project results and figures derived from the archived data are attributed to [11]; photographs and interface or pose screenshots are attributed to [10].